

Software-as-a-Graph: Heterogeneous Graph Learning for Pre-Deployment Dependability Analysis of Complex Distributed Systems

Ibrahim Onuralp Yigit^{a,*}, Feza Buzluca^a

^a*Department of Computer Engineering, Istanbul Technical University, Istanbul, 34469, Turkey*

Abstract

Asynchronous publish–subscribe and microservice architectures pose a pre-deployment visibility barrier: bug-free service code can still harbor catastrophic outages from hidden single points of failure or mismatched middleware contracts. We present Software-as-a-Graph (SaG), a static framework converting Architecture-as-Code manifests into typed multigraphs, paired with an interpretable ISO/IEC 25010 attribution layer. We evaluate a Heterogeneous Graph Transformer with Quality-of-Service edge encodings against homogeneous learning and training-free centrality across twelve inductive folds and five open-source systems, scored against simulation oracles.

The headline result is a boundary rather than a gain. Relation typing and QoS edge encoding each lift out-of-distribution rank correlation substantially when applied alone — $\Delta\rho = +0.234$ (12 of 12 folds, Holm-corrected $p = 0.002$) and $+0.287$ ($p = 0.003$) over an untyped, unweighted baseline — but combining them adds little: typing contributes $+0.035$ ($p = 0.129$) once QoS encoding is present, and QoS encoding $+0.087$ ($p = 0.204$) once typing is. The two are substitutes rather than complements; the framework’s own oracle suggests why, being 96.5% rank-recoverable with no QoS term at all. Against training-free QoS-weighted centrality the learned model shows no significant advantage ($+0.085$, $p = 0.151$), and zero-shot transfer to real systems falls from $\rho = 0.767$ overall to $+0.265$ on components that actually propagate failures, inverting on two of five. Inference costs 56 ms, but the deterministic feature extraction it needs is roughly eleven times more expensive than the simulation it was meant to displace. We report these limits as the contribution.

Keywords: Heterogeneous graph neural networks, Distributed systems dependability, Publish–subscribe architecture, Cascading failures, Static system analysis, Explainable AI

1. Introduction

1.1. Motivation

Modern large-scale distributed software systems increasingly rely on asynchronous, event-driven, and publish–subscribe (pub-sub) architectures. Across diverse domains—from autonomous driving (ROS [1]) and enterprise event streams (Apache Kafka [2]) to cyber-physical backbones (DDS [3]), IoT fleets (MQTT [4]), cloud-native microservices [5, 6], and distributed AI/LLM serving clusters—pub-sub decouples producers and consumers in space, time, and synchronization [7]. Components interact indirectly through intermediate message topics and brokers without maintaining direct static references. Furthermore, modern middleware specifications allow engineers to configure deployment-time Quality-of-Service (QoS) policies—such as reliability guarantees, durability, message priorities, and delivery deadlines—to govern how traffic behaves under peak load and network stress.

*Corresponding author

Email addresses: yigiti@itu.edu.tr (Ibrahim Onuralp Yigit), buzluca@itu.edu.tr (Feza Buzluca)

While this decoupling confers elastic scalability, it creates a **visibility barrier**. Where synchronous architectures (RESTful HTTP, gRPC) expose interaction as explicit caller–callee paths, pub-sub publishers and subscribers share no direct references: cascading failures, head-of-line blocking and backpressure travel hidden logical paths through brokers, shared topics, colocated hosts and shared libraries [8, 9]. Those disturbances also propagate by two different mechanisms — *sequential cascades*, where a slow subscriber saturates a broker queue and throttles its publishers hop by hop [10], and *simultaneous blast radii*, where a shared library crash or host outage disables every colocated service at once. Conventional architecture diagrams and static call graphs represent neither.

Addressing these vulnerabilities is most effective **prior to deployment**, during design and continuous integration, in keeping with the foundations of dependable computing [11, 12]. But at design and build time **no runtime telemetry, distributed tracing, or operational logs exist**. Architects and Site Reliability Engineers therefore face two questions with no operational data: which components, topics and links are systemically critical; and *why* each is critical, and which specific repair — replicating a broker, decoupling an over-subscribed topic, sandboxing a shared library — removes the risk.

The same questions bear on computational sustainability, though we are careful about the form of the claim. Analyzing an architecture from a manifest requires no provisioned cluster, no running services, and no fault-injection harness — the resource it eliminates is infrastructure rather than CPU time. What it does not do is compute less: the deterministic structural analysis this framework depends on is expensive, and on our own corpus it costs substantially more than the discrete-event simulation it produces labels from (Sections 7.5 and 8.2). We measure wall-clock latency rather than energy counters, and we make no claim that the pipeline is computationally cheap in absolute terms. The narrow and supported statement is that the *learned* component is negligible within it — a 56 ms forward pass against minutes of feature extraction — so applying graph neural networks to pre-deployment analysis does not itself introduce a meaningful cost.

1.2. Problem Statement: The Architecture–Code Gap and the Black-Box AI Challenge

We formulate pre-deployment dependability and performance analysis around two distinct, complementary tasks:

1. **Failure-Impact Forecasting (Predictive Pathway) — the primary task.** We forecast dynamic cascading failure blast radii and identify critical components using a data-driven, relation-specific model over learned topological representations. Closed-form topological metrics capture broad connectivity cheaply; whether they also resolve multi-hop, relation-dependent cascade spread across heterogeneous channels is an empirical question, which we test directly against such a baseline (Section 7.1). Our predictive pathway is trained and evaluated against independent simulation ground truth as a ranking and critical-set identification model.
2. **Explainable Criticality Attribution (Explanation Layer) — what a rank alone cannot say.** A ranked shortlist indicates *where* risk lies, but not *how to fix it*. We therefore pair the predictor with an interpretable structural quality profile grounded in ISO/IEC 25010 [13] and ISO/IEC 25019 [14]. This layer diagnoses the *qualitative root cause* of vulnerability—distinguishing, for instance, an unreplicated single point of failure from a high-coupling maintainability bottleneck—to guide concrete repairs. It serves strictly as an attribution model, not a ranking model.

This separation is architectural rather than merely presentational: both pathways operate on the same graph but share no parameters, and neither is trained on the other’s output. The coupling term that could connect them is disabled by default and reported only as an ablation (Section 4.2). Maintaining this independence allows SaG to identify components that are structurally central yet operationally low-impact—a nuanced diagnosis unattainable by either pathway alone.

The distance between an architecture as designed and as realised is long-established: Perry and Wolf [15] named architectural erosion and drift three decades ago, and the architectural-technical-debt literature has tracked it since. What we label the **Architecture–Code Gap** is a specialization of that idea to asynchronous middleware, where the problem is not that an implementation diverged from its design but that the design’s failure semantics were never expressible in the artifacts a build pipeline can read. Existing software engineering approaches do not bridge it: *a distributed system can have pristine, bug-free source code*

within each individual service, yet remain fragile to catastrophic global outages caused by hidden architectural single points of failure (SPOFs) or mismatched middleware Quality-of-Service (QoS) contracts. Classical architecture evaluation such as ATAM [16, 12], and the literature on architectural technical debt [17] and bad smells [18], identify architectural risks but rely on manual stakeholder elicitation rather than quantitative structural analysis. The automated paradigms each leave a different part of the gap unaddressed: static code analysis [19, 20, 21, 22] cannot see message queues or cross-host propagation; chaos engineering [23] needs a provisioned cluster and arrives after the architecture is fixed; and homogeneous centrality [24, 25, 26, 27] flattens the system into an untyped graph in which a topic, a library and a host are indistinguishable. Section 2 develops each in turn.

Furthermore, while machine learning has demonstrated remarkable success across software engineering, contemporary AI approaches applied to system dependability often function as uninterpretable black boxes. Deep neural models frequently output scalar risk scores or latent embeddings without providing transparent, actionable rationales for their predictions. In mission-critical software engineering, an opaque risk score is inadequate: developers and SREs cannot refactor code or reconfigure infrastructure without understanding *why* a component is vulnerable and *which* architectural mechanism is compromised.

1.3. The Software-as-a-Graph (SaG) Approach

To bridge the Architecture–Code Gap while overcoming the black-box AI challenge, this work introduces **Software-as-a-Graph (SaG)**, an AI-driven pre-deployment **Static System Analysis (SSA)** framework. SaG ingests Architecture-as-Code manifests and executes a four-stage pipeline:

1. **Typed Multigraph Formulation:** SaG models the distributed architecture as a typed, directed multigraph over five core entity types: Applications, Brokers, Topics, Execution Nodes, and Shared Libraries (Section 3.1).
2. **QoS-Aware Logical Dependency Projection:** Using six formal projection rules, SaG derives a semantic `DEPENDS_ON` dependency layer that captures both sequential cascades (via topics and brokers) and simultaneous blast radii (via shared libraries and node colocation), weighted by declared QoS contracts (Section 3.2).
3. **Heterogeneous Graph Learning for Failure Forecasting (Predictive Pathway):** SaG trains a **Heterogeneous Graph Transformer (HGT)** whose relation-specific attention lets a `USES` edge into a shared library propagate differently from a `PUBLISHES_TO` edge into a topic. It forecasts cascading blast radii, ranks critical components, and outputs per-relationship criticality alongside multi-task quality outputs (Section 4).
4. **Explainable Quality Attribution (Explanation Layer):** To explain *why* a flagged component is critical, SaG combines code-level SCA metrics with topological properties into a deterministic **Reliability–Maintainability (RM)** attribution model (Section 5). Reliability decomposes into **Fault Tolerance** (error propagation depth) and **Availability** (single-point-of-failure exposure), pointing to distinct repairs. Because it is a linear, propagation-free aggregate by design, it explains *why* a component is vulnerable rather than how far a cascade travels; its standalone rank correlation is correspondingly modest (Section 7.1).

To ensure methodological rigor, SaG enforces a strict **input–label independence guarantee**: learned models and attribution baselines operate exclusively on the analytical graph G_{analysis} , while ground-truth failure impacts are generated by independent discrete-event simulators operating on the raw structural topology $G_{\text{structural}}$ (Section 4.4).

Figure 1 shows how the two pathways relate. The predictive pathway is the primary one and the only pathway validated against the simulation oracle — the oracle scores rankings, which a quality profile is not — and that oracle is strictly an offline training-and-validation component, never a dependency of online inference. The explanation layer then characterizes what the predictor flagged and the remediation that implies. The single link between them is triage rather than data flow: the architect applies the explanation to whatever the predictor ranked. The remediation guidance closes a loop of its own in the Prescribe stage (Section 5.3), in which candidate edits are counterfactually re-simulated on mutated copies of $G_{\text{structural}}$ and retained only if they beat the simulator’s seed-to-seed noise.

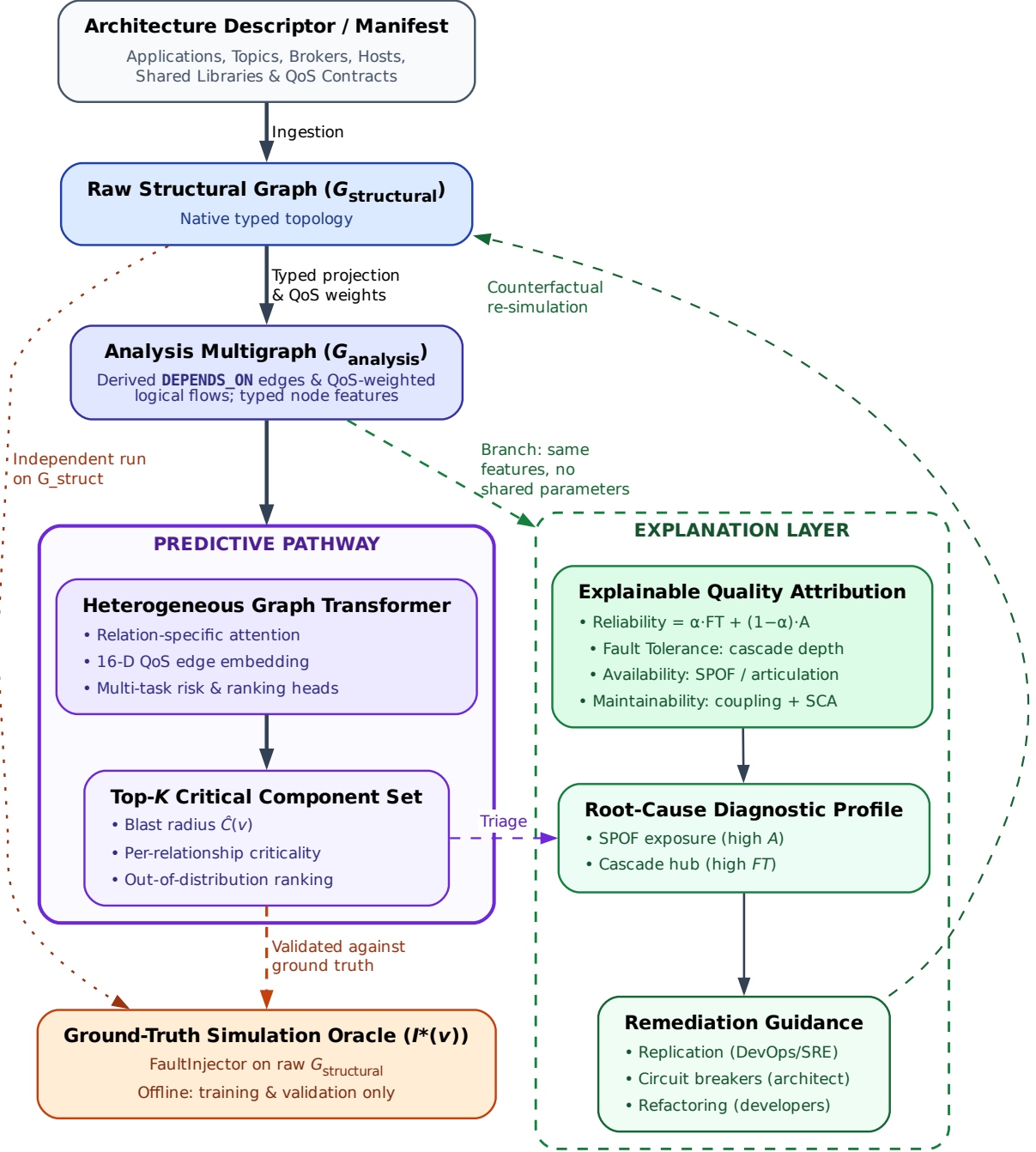


Figure 1: End-to-end architecture of the SaG framework. The predictive pathway (Section 4) is the centre line and runs straight down it: manifest ingestion $\rightarrow$ typed multigraph $\rightarrow$ QoS-weighted **DEPENDS_ON** projection $\rightarrow$ typed node features $\rightarrow$ heterogeneous graph learning $\rightarrow$ a ranked critical set with per-relationship criticality $\rightarrow$ the ground-truth simulation oracle (Section 4.3) that scores it. The oracle closes the predictive pathway’s training-and-validation loop and runs on $G_{\text{structural}}$ alone; it is offline and never a stage of inference (Section 4.4), which is why the edge into it is dashed. The explanation layer (Section 5) is the one branch off that line: it re-enters from the analysis multigraph, emits a standards-grounded quality profile from the same typed features while sharing no parameters with the predictor, and is reached by triage rather than by data flow.

Rationale for Graph Learning vs. Direct Simulation. Since discrete-event simulation $I^*(v)$ defines ground-truth criticality here, it is fair to ask why train a graph model at all. Two reasons survive our own measurements. A trained model scores entity types no simulator sweep was configured for, because message passing generalizes across labeled and unlabeled entities alike; and it returns per-relationship criticalities, which a node-level sweep cannot express. Two further reasons we previously advanced do not survive. Cascade simulation is stochastic, but barely so on this corpus (label test–retest median 0.982), so there is little noise for a smooth surrogate to absorb. And the claim that simulators need runnable containers is false of the simulator we actually use: **FaultInjector** reads the same manifest and costs 0.14–7.2s per scenario, roughly eleven times less than the deterministic feature extraction the learned pipeline depends on (Section 7.5.1). The argument holds only against runtime fault injection on a provisioned cluster, a comparison we do not measure. Whether the two surviving motivations translate into empirical gains is tested in Section 7.1, which answers only partly in the framework’s favor.

1.4. Research Questions

This empirical study investigates five research questions:

RQ1 (Predictive Efficacy): *How accurately does heterogeneous graph learning predict cascading failure impact and identify the critical component set, compared with traditional, non-learning network metrics?*

RQ2 (Value of Architectural Typing): *Does modeling distinct entity and dependency types (applications, topics, brokers, hosts, and libraries) yield better failure predictions than homogeneous graph models, and does that advantage hold on architectures the model has never seen?*

RQ3 (QoS Encoding and Robustness): *Do middleware Quality-of-Service contracts carry signal a purely structural score discards, do the framework’s simulation oracles agree with one another, and are the reported orderings robust to the free parameters of the scorer and of the ground truth?*

RQ4 (Real-World Generalization): *How effectively does the framework transfer zero-shot to authentic, real-world distributed systems across autonomous driving (ROS 2), cloud-native microservices, smart home IoT, and industrial edge computing?*

RQ5 (Analysis Cost): *What does pre-deployment analysis cost at CI/CD time, which pipeline stage dominates that footprint, and how does it compare against the discrete-event simulation it is intended to displace?*

1.5. Key Contributions

This paper presents four principal contributions:

1. **Heterogeneous Graph Learning for Pre-Deployment Dependability, and Its Limits:** A relation-specific Heterogeneous Graph Transformer that forecasts cascading blast radii from Architecture-as-Code manifests, with a 16-D edge feature vector carrying 7 QoS dimensions and multi-task heads for component and relationship criticality (Section 4). Ablated separately under inductive distribution shift across twelve architectures, relation typing is worth $\Delta\rho = +0.234$ over an untyped, unweighted baseline (12 of 12 folds, Holm-corrected $p = 0.002$) and the QoS edge encoding $+0.287$ ($p = 0.003$) — but the two do not compose: each contributes little once the other is present ($+0.035$, $p = 0.129$; and $+0.087$, $p = 0.204$). Against an unparameterized QoS-weighted centrality baseline, learned ranking is not significantly better ($+0.085$, $p = 0.151$). We report the non-composition as the finding (Sections 7.1–7.2).
2. **A Formal Typed Architecture Model:** A multigraph representation that derives logical dependencies from physical pub-sub linkages and distinguishes sequential cascade propagation from simultaneous multi-consumer library failures (Section 3).

3. **A Standards-Grounded Explanation Layer:** An interpretable Reliability–Maintainability model grounded in ISO/IEC 25010/25019 that turns ranked predictions into actionable diagnoses, separating single-point-of-failure exposure from error-propagation reach (Section 5).
4. **Empirical Benchmark, Real-World Transfer, and Cost Profile:** An evaluation across twelve synthetic topologies (2,461 components) and five open-source systems (351 components) under strict graph-view separation. We characterize pipeline cost, showing that the neural model represents only 0.02% of runtime, while deterministic feature analysis dominates (82.7 s vs. 7.2 s for simulation), leading us to withdraw the conference version’s computational-efficiency claim (Sections 6–7).

Relationship to the authors’ prior work. An earlier conference paper [28] introduced the preliminary multi-graph formulation and deterministic quality model on synthetic topologies. This JSS manuscript substantially extends that work by introducing: the entire predictive HGT pathway with 16-D QoS edge encoding and multi-task heads (Section 4); inductive LOSO cross-validation (Section 7.2); zero-shot evaluation across five open-source systems (Section 7.4); empirical cost and sustainability characterization (Section 7.5); multi-oracle convergent validity and graph-view separation (Sections 4.3–4.4); and global sensitivity analyses (Section 7.3). Retained formalisms from the conference paper are limited to restructured portions of Sections 3 and 5.

1.6. Paper Organization

The remainder of this paper is organized as follows: Section 2 reviews related work. Section 3 formalizes the SaG multigraph model and dependency projections. Section 4 details the Heterogeneous Graph Transformer and simulation oracles. Section 5 presents the ISO/IEC-grounded explanation layer. Section 6 outlines the experimental methodology, while Section 7 reports empirical results for RQ1–RQ5. Section 8 discusses practical implications, sustainability, threats to validity, and limitations. Section 9 concludes.

2. Related Work

This work builds upon and connects four foundational research areas: (1) dependability, performance, and sustainability in distributed software systems; (2) static code and system analysis; (3) software quality measurement and multi-criteria evaluation; and (4) graph representation learning and explainable AI (XAI).

2.1. Dependability, Performance, and Sustainability in Distributed Software Systems

The publish–subscribe (pub-sub) and asynchronous event-driven paradigms decouple communicating entities in space, time, and synchronization, enabling elastic scalability and high throughput [7]. Modern middleware standards—such as ROS 2 [1], Apache Kafka [2], DDS [3], and MQTT [4]—govern these exchanges through fine-grained Quality-of-Service (QoS) policies that regulate message durability, transport reliability, priorities, and delivery deadlines. In cloud-native microservice meshes and distributed AI/LLM serving backbones, asynchronous message passing and queueing topologies form the primary communication substrate, directly shaping tail latencies, throughput bottlenecks, and hardware resource utilization.

Prior dependability and performance research has focused predominantly on **runtime mechanisms**, including dynamic consensus protocols, broker clustering, adaptive backpressure throttling, autoscaling, and automated failover. In parallel, **chaos engineering and runtime verification** [23] inject faults or latency into staging or production clusters to observe degradation and recovery. While runtime fault injection delivers operational validation that no static method can match, it requires a fully provisioned cluster, carries the risk of real service disruption, and consumes cluster-hours per sweep — which places it, alongside model training, among the development-time computations whose energy cost green software engineering has argued should be accounted for rather than assumed away [29, 30, 31]. In practice this precludes its use during architectural design or lightweight commit-level CI/CD.

Our work addresses the complementary **pre-deployment phase**: predicting systemic cascading vulnerabilities and performance degradation directly from Architecture-as-Code descriptors before runtime infrastructure is provisioned. From a green software engineering perspective, the input is a manifest rather

than an active deployment. We are careful about how far this argument reaches: it is a claim about what must be provisioned, not that the analysis uses less computation than alternatives—a distinction our measurements force, since the static gate proves more expensive than the simulation oracle it was intended to displace (Sections 7.5.1 and 8.2). Avoiding production restart storms remains motivating rather than an empirical claim.

Architecture-Based Reliability Prediction. Predicting dependability from an architectural description is not a new ambition, and SaG should be read against the tradition that pursued it analytically. Cheung’s absorbing-Markov-chain model [32] derives system reliability from component reliabilities and a transfer-of-control graph; Goseva-Popstojanova and Trivedi [33] systematize the state-based, path-based and additive families that followed, and Immonen and Niemelä [34] survey them from the architectural perspective. Model-driven descendants such as the Palladio Component Model [35] and layered queueing networks [36] predict performance and reliability from parameterized component models, and annotation-based methods such as the AADL Error Model Annex [37] let architects declare component error states and propagation paths to generate fault trees automatically. Where authored, all of these answer strictly richer questions than ours. The distinction is in required inputs: they need deliberate, pre-calibrated failure semantics — per-component failure probabilities, transition rates, operational profiles — that are unavailable at commit time without operational telemetry. SaG asks a narrower question in exchange: given only declared deployment manifests, which components’ failures would propagate furthest through the declared topology? Where those parameters can be obtained, an analytical model answers a stronger question than a ranking does, and we make no claim to displace it.

Data-Driven Failure Prediction and Root-Cause Analysis in Microservices. A large recent literature localizes faults in microservice systems from operational data: Seer [38] and Sage [39] predict and debug QoS violations from traces and hardware telemetry; MicroRCA [40] and TraceRCA [41] localize root causes over service-dependency and trace graphs; DeepTraLog [42] and Eadro [43] combine traces, logs and metrics under graph-based deep models. Zhang et al. [44] survey 98 papers in this space and organize it by the observability modality each method consumes, which is the axis that matters here. This is the closest methodological neighbor to our predictive pathway, and it consistently outperforms what a purely static analysis can achieve — because it observes the running system. That is precisely the boundary: every one of these approaches requires a deployed system emitting traces, logs or metrics, and therefore cannot answer a question posed at design or pull-request time. SaG occupies the pre-deployment complement, and accepts a correspondingly weaker evidential basis: simulated rather than observed failures, and topology rather than behavior.

2.2. Static Code Analysis (SCA) vs. Static System Analysis (SSA)

Traditional **Static Code Analysis (SCA)** tools (e.g., SonarQube [19]) inspect source code Abstract Syntax Trees (ASTs) within individual services. They evaluate cyclomatic complexity [20], class cohesion, module coupling (e.g., Lack of Cohesion in Methods [LCOM], Coupling Between Objects [CBO]) [21, 22], and code duplication to flag internal code smells and defect-prone modules [45, 46, 47, 48]. However, SCA cannot observe runtime communication topology: it is blind to inter-service messaging channels, message broker queue saturation, and cross-host failure propagation.

Recovering system-level structure statically is, however, an active area in its own right, and we do not claim the idea as novel. A body of work reconstructs microservice architecture from source and deployment artifacts without running the system: Bushong et al. [49] derive communication diagrams and bounded contexts from static code analysis of a service mesh, and a recent multivocal review compares nine such recovery tools and finds their outputs complementary enough that combining them improves detection [50]. That literature and ours differ in what the recovered graph is *for*: architecture recovery aims to reproduce a faithful description of the system as built, typically for comprehension or drift detection, whereas we take a declared topology as given and ask which of its components a failure would propagate furthest from. Recovery is, in that sense, an upstream complement — it could supply the manifests SaG consumes for a system whose Architecture-as-Code description is incomplete.

To bridge this “Architecture–Code Gap,” **Static System Analysis (SSA)** extends static analysis from single-service source code to the global system architecture. By modeling distributed applications, message topics, brokers, execution nodes, and shared libraries as a connected multigraph, SSA propagates code-level quality metrics across architectural dependencies. This allows engineering teams to detect structural anti-patterns [51, 52] and architectural technical debt [53] early during continuous integration (CI/CD) [54, 55], before defective topologies enter production.

2.3. Software Quality Models and Multi-Criteria Evaluation

Software product quality is standardized by the **ISO/IEC 25010:2023** product quality model [13] and the **ISO/IEC 25019:2023** Quality-in-Use model [14]. ISO/IEC 25010:2023 defines three closely intertwined characteristics critical to modern distributed systems:

- **Reliability:** The degree to which a system performs specified functions under stated conditions, comprising Faultlessness, Availability, Fault Tolerance, and Recoverability.
- **Maintainability:** The degree of effectiveness and efficiency with which software can be modified, comprising Modularity, Reusability, Analyzability, Modifiability, and Testability.
- **Performance Efficiency:** Performance relative to resource consumption under stated conditions, comprising Time Behavior (latency, response time), Resource Utilization (CPU, memory, bandwidth), and Capacity.

SaG operationalizes a strict subset of these: Availability and Fault Tolerance under Reliability, and Modularity, Modifiability, and Analyzability under Maintainability (Section 5.1). Faultlessness, Recoverability, Reusability, and Testability are not derivable from deployment topology alone and are outside the scope of this work.

Software engineering measurement explicitly distinguishes between *internal quality* (measured on static artifacts at rest) and *external quality* (measured on executing software systems) [56, 57]. In distributed architectures, architectural debt (such as over-centralized message topics or unreplicated brokers) degrades internal quality and precipitates severe external performance bottlenecks, queue congestion, and outages.

Aggregating multi-attribute structural metrics into an auditable quality score constitutes a classic Multi-Criteria Decision Making (MCDM) problem. The **Analytic Hierarchy Process (AHP)** [58] delivers a structured pairwise-comparison method with an explicit Consistency Ratio ($CR \leq 0.10$) intended to certify that elicited judgments are mutually coherent. That statistic detects *inconsistency*; it cannot detect a matrix filled in from an answer already chosen, which is a limitation we take seriously for our own weights and quantify in Supplementary Section S4. This study applies AHP to construct an audited, explainable Reliability–Maintainability (RM) quality baseline, in conjunction with learned graph models.

2.4. Graph Representation Learning and Explainable AI

Network science provides established centrality metrics to identify critical nodes, including degree, closeness, betweenness centrality [24, 26], articulation points, and PageRank [25, 27]. Foundational studies on network robustness [10], cascading overloads [8], and interdependent networks [9] model disruption propagation across connected topologies. While percolation models offer natural comparators, our training-free baselines are centrality-based (Section 6.2); evaluating targeted percolation fragmentation remains a recognized future baseline comparison.

However, standard network metrics suffer from two major limitations when applied to software architectures: (1) **Dimensional Collapse**, where a single centrality scalar cannot distinguish *why* a component is critical (e.g., an isolated single point of failure vs. an error-propagating cascade hub vs. an over-shared library); and (2) **Semantic Collapse**, where unweighted metrics treat all nodes and edges identically, conflating fundamentally different architectural entities such as asynchronous message topics, shared libraries, and physical execution hosts.

To overcome hand-engineered metrics, recent studies apply machine learning to network vulnerability (e.g., FINDER [59], DrBC [60], PowerGraph [61]). However, most models rely on **homogeneous message**

passing (GCN [62], GraphSAGE [63], GAT [64]), averaging signals indiscriminately across connection types. Because distributed software architectures are inherently **heterogeneous**, homogeneous models blur entity boundaries and fail to generalize out-of-distribution. Heterogeneous Graph Neural Networks (RGCN [65], HAN [66], HGT [67], MAGNN [68]) resolve this via relation-specific transformations. We build upon the **Heterogeneous Graph Transformer (HGT)** [67] to preserve typed relational semantics when forecasting cascade blast radii. Graph learning has been applied to microservice topologies directly — Khodabandeh et al. [69] predict future service interactions with graph attention over temporally segmented call graphs — but that work forecasts *which edges will exist* from observed interaction history, whereas we take a declared topology as given and forecast the blast radius of removing a node from it.

Explainable AI (XAI) vs. The Black-Box Barrier. A critical hurdle in applying modern AI to software engineering is the **black-box barrier**: deep neural models output risk scores or continuous embeddings without explaining underlying structural causality. In production software engineering, uninterpretable risk rankings hinder actionable decision-making: developers and SREs cannot determine whether to replicate a host, configure circuit breakers, or refactor shared libraries.

Existing GNN explanation techniques, such as GNNExplainer [70] and PGExplainer [71], identify influential subgraphs through edge masking or parameterized learning. Although useful, these methods explain the model using internal latent representations rather than standardized software engineering concepts. SaG resolves this limitation through a decoupled dual-pathway design: the predictive HGT pathway reveals typed mutual-attention distributions indicating *which* architectural relations propagated the cascade (Section 7.3.3 and Supplementary Section S8), while the deterministic explanation layer attributes fragility to standardized ISO/IEC quality sub-characteristics (Section 5), translating raw predictions into actionable, cost-effective remediations.

3. The Software-as-a-Graph (SaG) Architectural Model

This section formalizes the Software-as-a-Graph multigraph representation (Section 3.1), the QoS-aware weighting and logical dependency derivation rules (Section 3.2), the dual graph views (Section 3.3), and the typed node feature encodings (Section 3.4).

3.1. Formal Multigraph Definition

A complex distributed software system is formally modeled as a typed, weighted, directed multigraph:

$$\mathcal{G} = (V, E, \tau_V, \tau_E, w_V, w_E) \quad (1)$$

where:

- V is the set of system entities, partitioned into five disjoint entity types:

$$V = V_{\text{app}} \cup V_{\text{broker}} \cup V_{\text{topic}} \cup V_{\text{node}} \cup V_{\text{lib}} \quad (2)$$

- E is the set of directed edges connecting entities.
- $\tau_V : V \rightarrow \mathcal{T}_V$ and $\tau_E : E \rightarrow \mathcal{T}_E$ are typing functions assigning node and edge categories.
- $w_V : V \rightarrow [0, 1]$ and $w_E : E \rightarrow [0, 1]$ are weighting functions representing entity criticality and connection strength.

Table 1 summarizes the five entity types and six structural edge types formalized in the SaG model, along with their semantics and representative concrete distributed-system implementations.

Application and Library entities additionally incorporate static code metrics computed via Static Code Analysis (SCA) tools (`cm_*` attributes: lines of code, cyclomatic complexity, coupling between objects, LCOM), linking code-level fragility directly to topological analysis.

Table 1: Entity and structural edge types in the SaG model.

Entity Type ($\mathcal{T}_V$)	Architectural Role	Concrete System Examples
Application (V_{app})	Autonomous process producing/consuming messages	ROS 2 node, Kafka microservice, MQTT client
Broker (V_{broker})	Message routing and queuing intermediary	RabbitMQ exchange, Mosquitto, EMQX broker
Topic (V_{topic})	Named logical communication channel	/sensor/lidar, orders.payment.completed
Node (V_{node})	Physical host or virtualized execution environment	Bare-metal server, Kubernetes worker, Cloud VM
Library (V_{lib})	Shared software package or runtime dependency	librdkafka, OpenCV, Protobuf runtime
Structural Edge ($\mathcal{T}_E$)	Direction	Semantic Meaning
PUBLISHES_TO	App/Library $\rightarrow$ Topic	Component publishes messages to topic
SUBSCRIBES_TO	App/Library $\rightarrow$ Topic	Component consumes messages from topic
ROUTES	Broker $\rightarrow$ Topic	Broker manages and routes topic traffic
RUNS_ON	App/Broker $\rightarrow$ Node	Process is hosted on physical/virtual host
CONNECTS_TO	Node $\rightarrow$ Node	Physical network link between hosts
USES	App $\rightarrow$ Library	Application links to shared library dependency

3.2. QoS-Aware Weights and Logical Dependency Derivation

In distributed middleware, communication links vary in coupling strength based on their Quality-of-Service (QoS) contracts. For instance, a **RELIABLE** topic with **TRANSIENT_LOCAL** durability binds communicating services substantially more tightly than a **BEST_EFFORT** telemetry stream.

Each topic t carries an intrinsic criticality weight $w(t) \in [0, 1]$ combining its declared QoS semantics with two runtime-stress modulators: payload size and publication frequency:

$$w(t) = \beta \cdot \text{QoS}(t) + \alpha \cdot \text{SizeNorm}(t) + \psi \cdot \text{FreqNorm}(t), \quad (\beta, \alpha, \psi) = (0.75, 0.15, 0.10) \quad (3)$$

where the QoS term is an AHP-weighted aggregate of the declared contract:

$$\text{QoS}(t) = w_{\text{rel}} \cdot q_{\text{rel}} + w_{\text{dur}} \cdot q_{\text{dur}} + w_{\text{prio}} \cdot q_{\text{prio}}, \quad (w_{\text{rel}}, w_{\text{dur}}, w_{\text{prio}}) = (0.24, 0.62, 0.14) \quad (4)$$

Here, $q_{\text{rel}}, q_{\text{dur}}, q_{\text{prio}} \in [0, 1]$ represent normalized reliability, durability, and transport-priority scores. Durability dominates because it governs whether data persists across restarts and network partitions. Reliability and transport priority both govern in-flight delivery quality, with reliability receiving higher weight because unconditional delivery guarantees precede message scheduling. The sub-weight vector is the geometric-mean priority vector of an independently stated Saaty pairwise-comparison matrix, printed with its consistency computation in Supplementary Section S4; $CR = 0.016$, small but non-zero. The modulators are logarithmically compressed and clamped to $[0, 1]$: $\text{SizeNorm}(t) = \min(1.0, \log_2(1 + \text{bytes})/20)$ (a 1 MiB design envelope, representing the practical DDS sample ceiling before RTPS fragmentation dominates) and $\text{FreqNorm}(t) = \min(1.0, \log_{10}(1 + \text{Hz})/3)$. The final weight $w(t)$ is clamped to $[0.01, 1]$, ensuring that best-effort edges remain visible to graph traversals. Every structural communication edge incident on t (**PUBLISHES_TO**, **SUBSCRIBES_TO**, **ROUTES**) inherits $w_E(e) = w(t)$ along with the topic's QoS vector.

The outer split (β, α, ψ) is a declared convex combination. Sweeping it over the full simplex changes the induced ordering of $w(t)$ by at most $\rho = 0.081$ and downstream rank correlation by at most 0.031, so it is a documented convention rather than a tuned parameter (Supplementary Section S1).

3.2.1. Logical Dependency Projection (**DEPENDS_ON**)

Structural edges capture explicit deployment connections but omit implicit runtime dependencies. For example, a subscriber depends upon a publisher, yet no direct edge connects them in pub-sub architectures. We therefore derive a single unified semantic relation, **DEPENDS_ON**, directed from *dependent* to *dependency* (“if target fails, source is impacted”), according to the six projection rules detailed in Table 2:

Rules 1 and 2 aggregate the set of topics T connecting a component pair using a probabilistic union rather than a maximum [72, 73, 74]. This guarantees that additional parallel failure vectors increase coupling monotonically while keeping $w \in (0, 1]$. Rule 5 applies the harmonic mean $H(x, y) = 2xy/(x + y)$ [75] to combine the consuming Application's and the shared Library's vertex weights, balancing caller and dependency criticality. Rules 3 and 4 assign the maximum weight among component-level dependencies crossing the host boundary.

Table 2: The six `DEPENDS_ON` logical dependency projection rules.

Rule	Dependency Category	Structural Pattern (Dependent $\rightarrow$ Dependency)	Derived Weight (w)
1	<code>app_to_app</code>	Subscriber $\rightarrow$ Publisher (via shared Topic, incl. transitive <code>USES</code>)	$1 - \prod_{t \in T} (1 - w(t))$
2	<code>app_to_broker</code>	Publisher/Subscriber $\rightarrow$ Broker routing its topics	$1 - \prod_{t \in T} (1 - w(t))$
3	<code>node_to_node</code>	Host $\rightarrow$ Host (lifted from inter-host app dependencies)	Lifted $\max w$
4	<code>node_to_broker</code>	Host $\rightarrow$ Broker (lifted from hosted app dependencies)	Lifted $\max w$
5	<code>app_to_lib</code>	Application $\rightarrow$ Shared Library it <code>USES</code>	$H(w_V(\text{app}), w_V(\text{lib}))$
6	<code>broker_to_broker</code>	Broker $\leftrightarrow$ Broker (shared physical fault-domain colocation, symmetric)	$w_V(\text{node})$

3.2.2. Sequential Cascades vs. Simultaneous Blasts

A foundational principle of the SaG model is distinguishing between two fundamentally different degradation modes:

- **Sequential Cascade (Rule 1):** When an application publisher fails, downstream subscribers suffer message starvation. The failure propagates hop by hop through message queues and topic buffers.
- **Simultaneous Blast (Rule 5):** When a shared software library or execution node crashes, all consuming applications and colocated brokers fail *instantaneously* in a single shared-fate event.

Preserving architectural entity types and relation-specific projection rules enables SaG to model both mechanisms, whereas untyped homogeneous graphs collapse them into indistinguishable edges.

Rule 6 is intentionally the only symmetric projection rule. It does not imply that one broker functionally depends on another, but rather that two brokers colocated on the same host share that host’s physical failure domain. This follows the same simultaneous-blast principle as Rule 5, which is why the derived weight equals the shared Node’s weight and the relation is bidirectional. In production middleware deployments, colocated brokers compete for host resources (CPU cores, page cache, file descriptors, and NIC bandwidth); a host outage takes down all colocated instances simultaneously. Operational best practices for Kafka, RabbitMQ, and EMQX recommend distributing brokers across fault domains. Rule 6 does not model directional intra-cluster broker coupling (e.g., partition replication, controller quorum election, federation, or shovel links), which do not require physical colocation; extending the schema to capture these interactions is reserved for future work. Rule 6 applies in four of the eight scenarios forming the detection benchmark subset — the seven core synthetic topologies plus the ATM case study, the subset identified in Supplementary Section S12 — and contributes only 12 directed edges across them. Because the simulation oracles operate strictly on $G_{\text{structural}}$ (Section 4.4), Rule 6 has zero influence on ground-truth failure labels.

3.3. Dual Graph Views and Architectural Layers

The SaG framework maintains two distinct representations of the system:

1. **Structural Graph ($G_{\text{structural}}$):** The raw deployment graph containing physical and structural relations (such as `PUBLISHES_TO`, `ROUTES`, `RUNS_ON`, and `USES`). Discrete-event simulators consume this view exclusively to execute unbiased failure injections (Section 4.3).
2. **Analysis Graph (G_{analysis}):** The projected graph containing derived `DEPENDS_ON` edges annotated with QoS weights and ingested SCA code metrics. All GNN feature representations, graph embeddings, and analytical metrics are computed on G_{analysis} .

Supplementary Figure S14.1 illustrates this duality on a running example, contrasting the raw structural graph against the derived `DEPENDS_ON` projection.

G_{analysis} is further structured into four analytical layers (Application, Middleware, Infrastructure, and Global System), enabling evaluation of criticality at subsystem levels, consistent with hierarchical frameworks such as MIL-STD-498 [76].

3.4. Typed Node Feature Encoding

Both pathways read the same typed node properties from G_{analysis} : the predictive pathway (Section 4) projects them per entity type before heterogeneous message passing, and the explanation layer (Section 5) aggregates them into its quality profile. All five entity types share indices 0–17, a common block of topological metrics — in/out degree, betweenness, closeness, reverse PageRank, clustering coefficient, articulation score and bridge load — produced by the deterministic analysis stage whose cost is characterized in Section 7.5. Type-specific blocks extend it to between 19 and 25 dimensions: source-code metrics and the Code Quality Penalty for Applications, two reverse-USES blast-radius drivers for Libraries, queue capacity for Brokers, publisher/subscriber counts and ordinal QoS criticality for Topics, and CPU and memory allocation for Infrastructure Nodes. Supplementary Section S11 gives the index-by-index schema.

That the shared block is where the graph structure lives matters for interpreting Section 7: betweenness, closeness, reverse PageRank and articulation score are already summaries of the topology, computed before any model sees the graph. A learned model is therefore not the only route from structure to a criticality score, which is what makes the closed-form baselines fair comparators rather than strawmen.

4. Graph Learning for Failure-Impact Prediction

Cascading failure impact in distributed software systems is inherently non-linear, multi-hop, and relation-dependent. Outages propagate not merely based on neighbor count, but through architectural relations and dependencies extending multiple hops beyond the initial fault. Whether a closed-form combination of standard centrality metrics can capture these compound dynamics is an empirical question rather than a settled one. The primary predictive pathway of Section 1.2 therefore employs a learned graph model, and Section 7.1 evaluates it against exactly such a closed-form baseline — which, on out-of-distribution ranking, it does not significantly surpass.

This section details the Heterogeneous Graph Transformer (HGT) architecture and its typed edge encodings (Section 4.1), the multi-task prediction heads and dimension-masked loss formulation (Section 4.2), the ground-truth simulation oracles (Section 4.3), and the input-label independence guarantee that prevents data leakage (Section 4.4).

4.1. Heterogeneous Graph Transformer Architecture

Because distributed systems comprise heterogeneous entity types (Applications, Libraries, Brokers, Topics, Infrastructure Nodes) and diverse interaction semantics (PUBLISHES_TO, SUBSCRIBES_TO, ROUTES, RUNS_ON, CONNECTS_TO, USES, DEPENDS_ON), we employ a three-layer **Heterogeneous Graph Transformer (HGT)** architecture [67], implemented within PyTorch Geometric [77], with hidden dimension $D = 64$ and $H = 4$ attention heads. This architecture ensures that typed relations, rather than simple adjacency, govern failure-impact forecasting.

4.1.1. Continuous-Categorical Edge Feature Encoding (16-D)

To capture continuous QoS constraints and channel semantics, SaG encodes each directed edge $e = (u, v)$ as a 16-dimensional continuous-categorical vector $e_{uv} \in \mathbb{R}^{16}$. Index 0 is the scalar coupling weight $w_E(e) \in (0, 1]$ of Section 3.2; index 1 is the normalized count of simple paths through e ; indices 2–8 one-hot encode the seven structural and derived relations; and indices 9–15 carry middleware QoS parameters on PUBLISHES_TO and SUBSCRIBES_TO edges, zeroed elsewhere. Six QoS dimensions are active in our corpus — reliability, durability, message priority, a heterogeneity flag raised when an edge’s QoS triple departs from its scenario’s modal profile, and the deadline pair (an active flag and $\log_{10}(1 + \text{deadline_ns}/10^6)$), populated on 463 of 615 topics, 75%). The seventh, $\log_{10}(1 + \text{max_blocking_ms})$, is a schema provision for hard real-time DDS and ROS 2 profiles and is zero throughout.

An edge projection module maps e_{uv} into the hidden space: $e'_{uv} = W_{\text{edge}}e_{uv}$. Prior to relational attention computation, this projection vector is incorporated directly into the target node representation: $\tilde{h}_v = h_v + e'_{uv}$.

4.1.2. Type-Specific Projection and Heterogeneous Message Passing

For each source node u and target node v connected by meta-relation $\tau(e) = (\tau(u), \phi(e), \tau(v))$:

1. **Type-Specific Projection:** Node feature vectors x_v (of dimension 19–25 depending on entity type $\tau(v)$) are mapped into the shared D -dimensional hidden space:

$$h_v^{(0)} = \text{LayerNorm}(\text{GELU}(W_{\tau(v)}x_v)) \quad (5)$$

2. **Relational Mutual Attention:** Type-parameterized Query (Q), Key (K), and Value (V) projections calculate relation-specific attention. For head $i \in \{1, \dots, H\}$, with the softmax taken over the incoming neighborhood $\mathcal{N}(v)$:

$$\text{Attn}^i(u, e, v) = \text{Softmax}_{u \in \mathcal{N}(v)} \left(K^i(u) W_{\text{att}, \phi(e)}^i Q^i(\tilde{h}_v)^\top \cdot \frac{\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}}{\sqrt{D/H}} \right) \quad (6)$$

where $\mu_{\langle \tau(u), \phi(e), \tau(v) \rangle}$ is the learned per-meta-relation scaling prior of Hu et al. [67], which lets the model weight an entire relation triple up or down independently of the node pair. We retain it: it is the parameter that most directly expresses “this relation type matters more than that one”, and the typing effect of Section 7.2 is what it exists to capture. The implementation is PyTorch Geometric’s HGTCConv [77], whose `p_rel` parameter is this term.

$$\text{Msg}(u, e, v) = V(u)W_{\text{msg}, \phi(e)} \quad (7)$$

3. **Bidirectional Message Passing:** To capture downstream consumer starvation and upstream back-pressure simultaneously, message passing is executed over both forward and transposed relation views (G_{analysis} and $G_{\text{analysis}}^\top$).
4. **Residual Aggregation and Layer Normalization:** Target node representations are updated across layers $l \in \{1, \dots, L\}$ via residual connections and layer normalization:

$$h_v^{(l)} = \text{LayerNorm} \left(h_v^{(l-1)} + \text{Dropout} \left(\sum_{u \in \mathcal{N}(v)} \text{Attn}(u, e, v) \cdot \text{Msg}(u, e, v) \right) \right) \quad (8)$$

Training Protocol and Optimization Hyperparameters. Models are optimized end-to-end using AdamW with initial learning rate $\eta = 3 \times 10^{-4}$, weight decay 10^{-4} , and dropout probability $p = 0.10$ applied post-attention. Learning rates follow a cosine annealing schedule with warm restarts (CosineAnnealingWarmRestarts, $T_0 = 75$, $T_{\text{mult}} = 2$, $\eta_{\text{min}} = 3 \times 10^{-6}$). Training executes for a maximum of 300 epochs with early stopping governed by a patience of 30 epochs monitored on validation loss over labeled nodes. Inductive subgraphs are processed per scenario using full-graph inductive packing without mini-batch subsampling, with validation masks isolating held-out nodes to prevent information leakage across data splits. Five independent random seeds {42, 123, 456, 789, 2024} are evaluated across all runs, redrawing both partition masks and initializations. *Selection protocol:* the architectural hyperparameters ($D = 64$, $H = 4$, dropout, learning rate, and schedule) follow values conventional for HGT [67]. The loss coefficients of Equation (9) have no such precedent — the objective is bespoke to this task — and were set by judgment and left untuned; we state this rather than appeal to a convention that does not exist for a five-term multi-task loss. Neither group was tuned against the in-distribution test split or the LOSO folds; no search over them was performed there. The real-world evaluation of Section 7.4.1 is a separate case and is documented separately: it runs at a different depth and epoch budget from every other learned result in this paper, and Section 7.4.1 states that configuration and how it was arrived at. This avoids selection leakage, at the cost of leaving open whether either family is reported near its own optimum — a comparison between untuned configurations, which we state rather than treat as a like-for-like optimum comparison.

4.2. Multi-Task Prediction Heads and Dimension Masking

From the final node embeddings $h_v^{(L)}$, SaG utilizes specialized multi-task prediction heads:

- **Reliability Head:** $\hat{R}(v) = \sigma(\text{MLP}_R(h_v)) \in [0, 1]$
- **Maintainability Head:** $\hat{M}(v) = \sigma(\text{MLP}_M(h_v)) \in [0, 1]$
- **Composite Failure Impact Head:** $\hat{I}^*(v) = \sigma(\text{MLP}_C(h_v \parallel \hat{R}(v) \parallel \hat{M}(v))) \in [0, 1]$
- **Relationship Criticality Head:** $\hat{Q}(u, v) = \sigma(\text{TypedEdgeEncoder}_{\phi(e)}(h_u, h_v, e_{uv})) \in [0, 1]$

4.2.1. Dimension-Masked Loss Formulation

The combined optimization objective integrates regression accuracy, multi-task dimension learning, ranking fidelity, pairwise ordering, and edge prediction:

$$\mathcal{L} = \mathcal{L}_{\text{composite}} + 0.5 \cdot \mathcal{L}_{\text{dimension}} + 0.3 \cdot \mathcal{L}_{\text{rank}} + 0.1 \cdot \mathcal{L}_{\text{pairwise}} + 0.3 \cdot \mathcal{L}_{\text{edge}} + \lambda_{\text{RM}} \cdot \mathcal{L}_{\text{consistency}} \quad (9)$$

where $I^*(v)$ is the simulated cascade impact defined by the primary oracle (Section 4.3), $\mathcal{L}_{\text{composite}} = \text{MSE}(\hat{I}^*(v), I^*(v))$, $\mathcal{L}_{\text{rank}}$ is the ListMLE listwise ranking loss [78] parameterized by temperature τ :

$$\mathcal{L}_{\text{rank}} = -\frac{1}{N} \sum_{i=1}^N \left(\frac{\hat{s}_{\pi_i}}{\tau} - \log \sum_{j=i}^N \exp \left(\frac{\hat{s}_{\pi_j}}{\tau} \right) \right) \quad (10)$$

where $\pi = (\pi_1, \dots, \pi_N)$ denotes the permutation of nodes sorted in descending order of ground-truth impact $I^*(v)$, and $\hat{s}_v = \hat{I}^*(v)$. At the baseline default $\tau = 1.0$, the formulation reduces to standard ListMLE; temperature parameter $\tau < 1.0$ is provided as a configurable hyperparameter for sharpening probability distributions over narrow prediction margins. Pairwise ordering fidelity is guided by margin-ranking loss $\mathcal{L}_{\text{pairwise}} = \frac{1}{|P|} \sum_{(u,v) \in P} \max(0, \gamma - (\hat{s}_u - \hat{s}_v))$ with margin $\gamma = 0.05$ over pairs $P = \{(u, v) \mid I^*(u) - I^*(v) > \gamma\}$, and $\mathcal{L}_{\text{consistency}} = \text{MSE}([\hat{R}(v), \hat{M}(v)]_{v \in \text{unlabeled}}, [R_{\text{RM}}(v), M_{\text{RM}}(v)]_{v \in \text{unlabeled}})$ regresses predicted heads toward the diagnostic pathway’s baseline (Section 5) on unlabeled nodes. Headline results use $\lambda_{\text{RM}} = 0$, guaranteeing that the predictive and explanatory pathways remain strictly independent.

Dimension Masking: Because dynamic cascade simulation ($I^*(v)$ via **FaultInjector**) observes runtime failure reachability rather than source-code maintainability, maintainability ground truth is unobserved during dynamic simulation. A separate change-propagation oracle $I_M(v)$ evaluates static structural change ripple at the Validate stage, but is never used as a training label to avoid circular supervision. We introduce a boolean dimension mask $m = [m_R, m_M] = [1, 0]$:

$$\mathcal{L}_{\text{dimension}} = \frac{1}{\sum_d m_d} \sum_{d \in \{R, M\}} m_d \cdot \text{MSE}(\hat{d}(v), d^*(v)) \quad (11)$$

This mask ensures the unobserved maintainability head is not artificially penalized or driven toward zero during backpropagation.

What the reliability label is. The surviving term deserves to be stated plainly, because it is weaker than a multi-task objective normally implies. **FaultInjector** emits a single scalar per component, and the label extractor assigns that same scalar to both the composite and the reliability columns: $R^*(v) = I^*(v)$ identically. $\mathcal{L}_{\text{dimension}}$ under $m = [1, 0]$ therefore regresses $\hat{R}$ toward the very target $\mathcal{L}_{\text{composite}}$ regresses $\hat{I}^*$ toward. The two terms are not redundant — they train separate heads, and $\hat{R}$ re-enters the composite head as an input ($\hat{I}^* = \sigma(\text{MLP}_C(h_v \parallel \hat{R} \parallel \hat{M}))$), so the term acts as an auxiliary path to the same supervision rather than as a second source of information. But no dimension of the label is decomposed by this oracle: a reliability score distinct from overall impact would require an oracle that separates fault-tolerance from availability, which $I^*(v)$ does not. We report the objective as implemented rather than describing it as multi-dimensional supervision it does not receive.

4.2.2. Domain-Reweighted Criticality

ISO/IEC 25019’s Context of Use implies that the weight placed on reliability against maintainability is a deployment choice rather than a constant, and the framework exposes a reweighting $Q_{\text{domain}}(v) = q_R \hat{R}(v) + q_M M_{\text{static}}(v)$ to express it. Because maintainability is unobserved under dynamic simulation ($m = [1, 0]$), no headline result uses it: every reported figure is $\hat{I}^*(v)$ directly. Supplementary Section S4 reports its sensitivity against the static RM baseline.

4.3. Ground-Truth Simulation Oracles

To evaluate predictive accuracy prior to deployment without relying on production runtime telemetry, SaG executes discrete-event failure simulations over the raw structural multigraph $G_{\text{structural}}$. We establish a formal taxonomy of four component-level oracles and one relationship-level oracle:

- **Cascade Reachability Oracle** ($I^*(v)$), via **FaultInjector**: crashes component v , propagates outages across dependent topics, brokers, and links by breadth-first traversal, and returns the mean fractional feed loss over the subscriber population, each subscriber contributing the unweighted mean loss of the topics it subscribes to. A topic’s feed loss is the fraction of its publishers that have failed (for a topic with no publisher, the fraction of its failed routers), scaled by a QoS ladder ($\times 1.2$ RELIABLE, $\times 1.15$ high/urgent priority, $\times 1.05$ medium) and clamped to $[0, 1]$. The denominator is the full subscriber set of the intact graph, so subscribers that themselves fail are retained in the average rather than excluded. The implementation admits a per-publisher rate weighting, but rates are declared per topic throughout our corpus, so it reduces exactly to this publisher fraction. This is the **primary continuous target label** throughout.

How much QoS is in this label. The ladder reads reliability and transport priority only; durability does not enter I^* at all, despite carrying the largest of the three QoS sub-weights in the framework’s own elicited vector (0.62, against 0.24 for reliability and 0.14 for priority; Section 3.2). That omission turns out not to be what limits the label’s QoS content. Re-running the labeler with QoS scaling disabled entirely leaves the Application ordering very nearly intact — mean Spearman $\rho = 0.965$ against the ladder across the twelve folds (range 0.891–0.999) — and substituting a durability-aware $w(t)$ scaling moves it less still ($\rho = 0.977$). Neither parameterisation materially reorders the target. The top- K critical set is the more sensitive construct: ladder and topology-only labels agree at mean Jaccard 0.678, so QoS does change *which* components are named critical without changing their order. I^* should therefore be read as a near-topological target that carries a QoS term at its threshold boundaries rather than through its ranking, which bounds what any QoS-encoding result can be crediting (Section 7.3.1).

- **Multi-Metric Composite Oracle** ($I_{\text{comp}}(v)$), via **FailureSimulator**: a severity-weighted blend of reachability loss, fragmentation, throughput loss and flow disruption, with AHP-derived coefficients (0.35, 0.25, 0.25, 0.15). Those coefficients come from a rank-one comparison matrix, so it records where they came from without independently justifying them, and they are not swept in our sensitivity analysis — a gap worth naming because I_{comp} supplies the labels for the explanation layer’s real-world evaluation (Supplementary Sections S4 and S7). It is reserved for Validate-stage gates and prescriptive verification, never for predictive ranking.
- **Dynamic Queue-Flow Oracle** ($I_{\text{dyn}}(v)$), via **MessageFlowSimulator** on SimPy [79]: simulates emission rates, stochastic latencies, broker buffer saturation, and queue drops under fault injection, extracting the drop in delivered message rate to surviving consumers. It serves only as an independent convergent-validity probe (Supplementary Section S9).
- **Change-Propagation Oracle** ($I_M(v)$), via **ChangePropagationSimulator**: a deterministic reverse-dependency traversal over the transpose of the six-rule DEPENDS_ON projection, blending change reach, weighted change impact and normalized depth. It is a structural maintainability reference and is never used as a training label, which would make the supervision circular.

- **Relationship (Edge) Removal Oracle** ($I_{\text{edge}}(u, v)$): the systemic impact of severing one dependency while both endpoints stay operational. Writing $\bar{I}_{\text{comp}}(G)$ for the mean composite impact over G :

$$I_{\text{edge}}(u, v) = \bar{I}_{\text{comp}}(G \setminus \{(u, v)\}) - \bar{I}_{\text{comp}}(G) \quad (12)$$

Topic Criticality Label Masking. `FailureSimulator` can blend a declared `Topic.criticality` into its severity term; this is disabled, because that field is a GNN input feature (`topic_qos_criticality_ord`) and consuming it would score the predictor against a transformation of its own input.

Primary Oracle Declaration and Role Assignment. Because the three reliability-facing oracles (I^* , I_{comp} , I_{dyn}) measure distinct operational constructs, we designate $I^*(v)$ (**FaultInjector**) as the **primary oracle** for all predictive ranking results (Tables 5–6, RQ1–RQ3). $I_{\text{comp}}(v)$ is reserved for Validate-stage quality gates and prescriptive remediation verification, $I_{\text{dyn}}(v)$ serves as an independent convergent-validity probe (Section 7.3.2 and Supplementary Section S9), and $I_M(v)$ serves as a structural maintainability reference.

Cross-Oracle Convergent Validity and Critical-Set Bounds. Measured across the twelve inductive folds on the Application population, the mean Spearman rank correlation is $\rho = 0.620$ for (I_{dyn}, I^*) , $\rho = 0.395$ for (I_{comp}, I^*) , and $\rho = 0.366$ for $(I_{\text{comp}}, I_{\text{dyn}})$. The substantial but sub-ceiling agreement ($\rho = 0.620$, against a label test–retest ceiling of 0.811–1.000) between the behavioral queue-flow oracle and the topological cascade injector provides independent convergent evidence across distinct simulation paradigms without collapsing into a re-measurement of the same construct. However, agreement on the top- K critical set ($K = 0.2n$) is more conservative (mean Jaccard overlap of 0.36 for the strongest pair and 0.27–0.28 for the two I_{comp} pairs, vs. 0.111 expected by chance), highlighting the intrinsic sensitivity of discrete thresholding in non-linear cascades. Consequently, results established against one oracle are never transferred to another; every evaluation metric explicitly references its underlying simulation oracle.

Seeding. These are twelve per-scenario comparisons, not twelve replicated at five seeds. The five seeds enter I^* alone, where they are averaged into one label per component; I_{comp} and I_{dyn} each run once, at seed 42. The artifact records this per oracle so the distinction cannot drift. What the pairs are bounded by is treated in Section 7.2.

4.4. Input–Label Independence Guarantee

To eliminate data leakage and ensure rigorous evaluation, SaG enforces strict architectural separation between inputs and labels:

- **Feature Space:** Constructed exclusively from G_{analysis} using static structural topology, static code analysis (SCA) metrics, and declared QoS contracts.
- **Label Space:** Evaluated exclusively on raw $G_{\text{structural}}$ through independent simulation oracles (**FaultInjector**, **FailureSimulator**, **MessageFlowSimulator**).

No simulation outputs, failure trace histories, or dynamic execution telemetry are ever exposed as input features to the GNN or the explanation layer.

What this guarantee does and does not establish. The separation rules out circular feature construction: no predictor can read a transformation of the quantity it is scored against. It does not establish statistical independence between features and labels, and we do not claim it does. G_{analysis} is a deterministic projection of $G_{\text{structural}}$ (Section 3.2), so the labels are — up to simulator seed — a deterministic function of the same topology the features are computed from. Two consequences follow, and both bound the results of Section 7.

First, $I^*(v)$ is itself a topological functional: a breadth-first reachability computation over $G_{\text{structural}}$ scaled by a QoS ladder. The predictive task is therefore to recover a closed-form function of a graph from features of that same graph. Read that way, it is unsurprising that an unparameterized centrality score is competitive with a trained model (Section 7.1); the parity we report there is the expected outcome of the setup rather than a surprising failure of graph learning, and we flag it here so the reader does not have to infer it.

Second, and more restrictively, no result in this paper is validated against an observed failure. Every label — on synthetic topologies and on the five open-source systems alike — is simulator-derived. What the evaluation can establish is whether a learned model recovers a simulator’s ordering on architectures it was not trained on. Whether that ordering corresponds to which components actually fail in production is a question this design cannot answer, and Section 8.3 treats it as the study’s principal construct-validity threat.

5. The Explanation Layer: Standards-Grounded Criticality Attribution

The predictor of Section 4 answers *where* to act. It does not answer *what to do*, and neither does the oracle that scores it — both return impact, not cause attributed in standardized quality terms. A component may be critical because it is a single point of failure, because it propagates errors widely, or because it is a high-coupling maintenance bottleneck. These diagnoses call for distinct architectural repairs: replicating the host or broker, decoupling the topic, or refactoring the module. This section presents the layer supplying that diagnosis. SaG decomposes component and relationship criticality into a standards-grounded quality profile, computed over the same typed node features (Section 3.4) but sharing no parameters with the predictor, and applied to whatever the predictor flagged — triage rather than data flow (Figure 1).

5.1. Grounding in ISO/IEC Standards

In accordance with **ISO/IEC 25010:2023** (Product Quality Model) [13], **ISO/IEC 25019:2023** (Quality-in-Use) [14], and **ISO/IEC 25022:2016** (Measurement of Quality-in-Use) [80], SaG formalizes two primary criticality constructs:

- **Component Criticality (D_1):** The degree to which the sudden failure, unexpected termination, or severe degradation of an individual component reduces the system’s capacity to deliver required services within its operational context of use.
- **Relationship Criticality (D_2):** The degree of systemic service degradation resulting from the severance, partitioning, or failure of a specific dependency or communication channel while both endpoint components remain operational.

Criticality is evaluated across two orthogonal quality characteristics: **Reliability (R)** and **Maintainability (M)**. The two are separated because they imply different repairs, not because they predict different runtime quantities: this layer attributes structural fragility and does not forecast latency, throughput, or queue occupancy. Table 3 outlines this Reliability–Maintainability (RM) quality decomposition, mapping ISO/IEC sub-characteristics to architectural questions, underlying graph metrics, and targeted engineering remediations.

Table 3: The Reliability–Maintainability (RM) quality decomposition.

Dimension	Sub-Characteristic	Architectural Question	Underlying Graph Metrics	Role / Remediation
Reliability (R)	Fault Tolerance (FT)	How broadly does failure propagate?	Reverse PageRank on G^T , in-degree, cascade depth	Reliability Eng.: add redundancy, circuit breakers
	Availability (A)	Is this a single point of failure?	Directed articulation score (raw + QoS-weighted), bridge ratio, connectivity degradation	DevOps/SRE: replicate host/broker
Maintainability (M)	Modularity/Modifiability	How complex and coupled is this?	Betweenness, QoS-weighted out-degree, Code Penalty, coupling-risk imbalance, inverse clustering	Architect: refactor code, decouple

Coverage Scope: SaG focuses specifically on Reliability and Maintainability. Safety (which requires domain-specific hazard logs, such as ISO 26262 Automotive Safety Integrity Level [ASIL] ratings) and Security (which requires explicit threat models, such as STRIDE [Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege]) fall outside purely structural topology analysis and are reserved for domain-specific extensions.

5.2. Composite Quality Score Formulation

All raw topological and code metrics are rank-normalized to $[0, 1]$. Quality sub-characteristics are formulated hierarchically using the Analytic Hierarchy Process (AHP) [58]:

1. **Fault Tolerance** ($FT(v)$): Measures error cascade potential on the transpose graph $G_{\text{analysis}}^\top$ (where edges follow failure propagation from dependency to dependent):

$$FT(v) = 0.45 \cdot \text{RPR}(v) + 0.30 \cdot \text{Deg}_{\text{in}}(v) + 0.25 \cdot \text{CDPot}_{\text{enh}}(v) \quad (13)$$

where $\text{RPR}(v)$ is Reverse PageRank, $\text{Deg}_{\text{in}}(v)$ is normalized in-degree, and $\text{CDPot}_{\text{enh}}(v)$ is the enhanced Cascade Depth Potential term.

2. **Availability** ($A(v)$): Identifies structural single points of failure (SPOFs) across five terms — directed articulation severity, its QoS-weighted variant, edge-level irrecoverability, connectivity degradation, and the component’s own QoS weight:

$$A(v) = 0.2563 \cdot \text{AP}_c^{\text{dir}}(v) + 0.1998 \cdot \text{QSPOF}(v) + 0.1998 \cdot \text{BR}(v) + 0.2563 \cdot \text{CDI}(v) + 0.0878 \cdot w(v) \quad (14)$$

where $\text{AP}_c^{\text{dir}}(v)$ is Directed Articulation Point severity, $\text{QSPOF}(v)$ is QoS-weighted Single Point of Failure severity, $\text{BR}(v)$ is Bridge Ratio (edge-level irrecoverability), $\text{CDI}(v)$ is Connectivity Degradation Index, and $w(v)$ is the component’s intrinsic QoS weight.

3. **Reliability** ($R(v)$): Blends Fault Tolerance and Availability hierarchically:

$$R(v) = r_\alpha \cdot FT(v) + (1 - r_\alpha) \cdot A(v), \quad r_\alpha = 0.36 \quad (15)$$

The blend weight $r_\alpha = 0.36$ is screened in Supplementary Section S1. The intra-dimension weights apply $\lambda = 0.70$ shrinkage blending with a uniform prior (Section 7.3). Because three of the comparison matrices are rank-one by construction (implying near-zero CR by construction, detailed in Supplementary Section S4), these weights represent declared constants with documented internal structure rather than independently elicited expert judgments.

4. **Maintainability** ($M(v)$): Evaluates structural coupling combined with code-level static analysis across five terms:

$$M(v) = 0.35 \cdot \text{BT}(v) + 0.30 \cdot w_{\text{out}}(v) + 0.15 \cdot \text{CQP}(v) + 0.12 \cdot \text{CouplingRisk}_{\text{enh}}(v) + 0.08 \cdot (1 - \text{CC}(v)) \quad (16)$$

where $\text{BT}(v)$ is Betweenness Centrality, $w_{\text{out}}(v)$ is QoS-weighted efferent coupling, $\text{CQP}(v)$ is Code Quality Penalty, $\text{CouplingRisk}_{\text{enh}}(v)$ is coupling imbalance, and $\text{CC}(v)$ is local Clustering Coefficient.

The baseline composite quality score $Q(v)$ combines both dimensions:

$$Q(v) = 0.80 \cdot R(v) + 0.20 \cdot M(v) \quad (17)$$

When evaluating under a specific ISO/IEC 25019 Context of Use vector $\vec{\omega} = [q_R, q_M]^\top$, the score is reweighted dynamically:

$$Q_{\text{domain}}(v) = q_R \cdot R(v) + q_M \cdot M_{\text{static}}(v) \quad (18)$$

Components are categorized into adaptive criticality tiers using Tukey quartile thresholds: **CRITICAL** ($Q > Q_3 + 1.5 \cdot \text{IQR}$), **HIGH** ($Q_3 < Q \leq Q_3 + 1.5 \cdot \text{IQR}$), **MEDIUM** ($Q_1 < Q \leq Q_3$), and **MINIMAL** ($Q \leq Q_1$). This yields an actionable distinction: high A with low FT indicates a single point of failure calling for horizontal replication, whereas high FT denotes an error-cascade hub requiring circuit breakers, rate limiting, and bulkhead isolation (Section 8.4).

5.3. Prescriptive Remediation and Counterfactual Verification

The attribution above names a repair; whether the repair helps is a separate question, and SaG answers it by re-simulating rather than by asserting. Once the explanation layer attributes a root cause, refactoring operators emit candidate repair manifests — replicating a single-point-of-failure broker, inserting a circuit breaker, decoupling a publisher–subscriber feed — and an **EditVerifier** builds the mutated graph G' in memory and re-runs multi-threshold cascade simulation on it. An edit is accepted only if it reduces systemic composite impact by more than the simulator’s own seed-to-seed noise ($\Delta \bar{I}_{\text{comp}} > \kappa \cdot \sigma_{\text{seed}}$, $\kappa \geq 1.0$) at every propagation threshold while creating no new articulation point; edge-level edits are verified against the

edge-removal oracle of Eq. (12). The acceptance rule matters more than the operators: a candidate repair that merely moves impact below one threshold is rejected.

We describe this stage because it closes the loop the explanation layer opens, and we make no empirical claim for it here — no prescriptive result is reported in this paper, and the operators are not evaluated against a baseline.

6. Experimental Setup

6.1. Datasets and System Corpus

The evaluation corpus comprises 2,812 components across seventeen system architectures — twelve synthetic topologies that form the inductive cross-validation folds, and five real-world reference systems withheld entirely from training — as detailed in Table 4:

Table 4: Evaluation corpus at a glance. The twelve synthetic topologies are the inductive Leave-One-Scenario-Out folds of Table 6; the five real-world systems are withheld from every training fold and used only for zero-shot transfer (Section 7.4). Per-scenario entity and edge counts, read from the committed topology files and verified against them in continuous integration, are in Supplementary Section S13.

Regime	$ V $	$ V_{\text{app}} $	Topics	Brokers	Hosts	Libs	$ E $
Synthetic evaluation scenarios (11)	2,387	1,295	588	60	194	250	10,657
Synthetic case study — ATM (1)	74	26	27	5	8	8	261
Synthetic subtotal (12 LOSO folds)	2,461	1,321	615	65	202	258	10,918
Real-world subtotal (5 systems)	351	141	120	16	32	42	700
Total	2,812	1,462	735	81	234	300	11,618

Here, $|V|$ is the sum of all five entity-type counts per scenario and $|E|$ counts every raw structural relationship instance in the scenario specification — the native substrate the simulation oracles traverse, across all six relation types — rather than the derived `DEPENDS_ON` projection built for GNN training. We note the convention because the generator’s own build log omits `CONNECTS_TO` and therefore reports lower per-scenario totals.

The five real-world architectures were transcribed from authentic open-source repositories using dedicated architectural adapters. The synthetic scenarios come from a parameterized topology generator: each is fully defined by a committed configuration specifying a random seed, per-entity-type counts, seven-number summaries for application publish and subscribe fan-out, applications per host, library fan-in and topic payload size, and categorical distributions over the three QoS dimensions. Degree distributions and clustering emerge from these parameters rather than being synthetically forced. Supplementary Section S5 reports the generative parameters per topology and Section S13 the resulting per-scenario counts; complete configurations ship in the replication package.

Corpus Composition across Experimental Regimes. The experimental evaluation spans three complementary regimes with precisely bounded corpora:

1. *In-Distribution Evaluation (Table 5)*: Evaluated across all twelve distributed architecture scenarios ($n = 12$) from Table 4 using stratified 60% train / 20% validation / 20% test node splits over five random seeds. This regime establishes the baseline fitting performance of each predictor when training and testing are drawn from the same underlying architectural distribution, complementing the inductive out-of-distribution evaluation.
2. *Inductive Leave-One-Scenario-Out (LOSO) Cross-Validation (Table 6)*: Evaluated across twelve distinct inductive folds totaling 2,461 components: the seven core synthetic scenarios, four extended domain topologies (Telecom RAN, Industrial SCADA, Real-Time Gaming, and Logistics Fleet) — 2,387 components between them — and an Air Traffic Management (ATM) network scenario contributing the remaining 74. In each fold, models are trained on eleven graphs and tested zero-shot on the held-out twelfth graph.

3. *Real-World Architectural Transfer (Table 9; Supplementary Section S7)*: The five open-source real-world systems (Autoware.universe, Cloud Microservices, Train-Ticket, Home Assistant, EdgeX Foundry) are never used as training folds; they are withheld entirely and used strictly for zero-shot architectural transfer validation.

Not every analysis in Section 7 runs on the full corpus: the sensitivity sweeps and the detection benchmark predate the four extended domains and operate on smaller cached subsets. Because a reader comparing figures across subsections would otherwise have no way to tell which population a number belongs to, Supplementary Section S12 tabulates the subset behind each analysis. Comparisons are made only within a row of that table, and never across rows.

6.1.1. Reproducibility of the Corpus

The benchmark corpus is designed to be fully regenerable rather than merely statically archived. Each dataset is deterministically generated from its configuration file via:

```
python cli/generate_graph.py batch -input-dir data/scenarios -output-dir <dir>
```

A companion manifest records the random seed, entity counts, git commit hash, and a SHA-256 cryptographic digest for each emitted topology. Continuous integration regression tests assert that every committed dataset regenerates *byte-identically* from its configuration and that all disk digests match the manifest. This guarantees that third parties can reproduce the exact graphs used in our experiments, rather than simply sampling from similar distributions.

6.2. Baselines and Evaluated Predictors

We evaluate four primary predictor configurations drawn from three families. Predictor names state the family and the substrate: an **-N** suffix marks a model trained on the *native* multigraph, its absence the derived Application-Library flow projection, and a **-QoS** suffix marks a configuration that consumes declared QoS contracts. *SaG* throughout denotes the framework, never an individual predictor.

1. **Heterogeneous graph learning (typed HGT). HGT-QoS** (proposed): relation-specific Heterogeneous Graph Transformer (Section 4) ingesting the complete native multigraph with 16-dimensional continuous-categorical edge features that encode middleware QoS contracts. Its ablation **HGT**, which masks those QoS dimensions, is reported in Section 7.3.1.
2. **Homogeneous graph learning (untyped GAT). GAT-N-QoS**: homogeneous Graph Attention Network [64] trained on the identical native multigraph substrate with per-type input projections, but untyped, single-relation message passing. Its edge channel carries the scalar QoS aggregate $w(e)$ — dimension 0 of the same 16-D encoding HGT-QoS consumes — rather than the per-dimension decomposition; no homogeneous architecture in our suite ingests the full 16-D vector. The HGT-QoS–GAT-N-QoS contrast therefore bounds the *joint* contribution of relational typing and per-dimension QoS encoding. Section 7.3.1 separates the second factor within the typed architecture, and the corresponding unweighted ablation is **GAT-N**. The **-N** suffix denotes the native substrate and is load-bearing: the same homogeneous architecture run on the **DEPENDS_ON** projection is reported as **GAT** / **GAT-QoS**, and that is the pair Table 5 carries.
3. **QoS-weighted structural baseline (training-free). Topo-QoS**: QoS-weighted topological centrality evaluated on the derived application flow projection.
4. **Unweighted structural baseline (training-free). Topo**: structural centrality combining unweighted betweenness centrality and articulation point scoring on the flow projection.

In addition, the out-of-distribution evaluation (Table 6) reports **RM** ($Q(v)$, the deterministic hierarchical quality attribution model of Section 5) as a diagnostic reference baseline. RM is not fitted to rank failure impact; its inclusion demonstrates how much learned relational prediction adds over static structural attribution (Section 1.2). Furthermore, deterministic RM scoring drives every sensitivity sweep in Section 7.3, where closed-form formulations isolate parameter effects from neural training stochasticity.

6.2.1. Evaluation Substrates and Parity Guarantee

Predictors in this study operate on matched substrates within their respective families:

- **Graph Learning Models (GAT-N-QoS, HGT-QoS):** Under Leave-One-Scenario-Out (Table 6) both learned predictors ingest the complete native typed multigraph across all five entity types — which is what the shared `-N` suffix records — so that comparison carries no multi-entity visibility confound. It does not hold in-distribution: the homogeneous pair reported in Table 5 (GAT / GAT-QoS, without the suffix) consumes the Application–Library `DEPENDS_ON` projection, so the in-distribution margin mixes typing with multi-entity visibility and we do not read it as a typing result. Node type reaches homogeneous GAT-N-QoS only through its per-type input projection layer, while message passing uses untyped GATConv with shared weights across all edges; in contrast, heterogeneous HGT-QoS employs relation-specific HGTCov weight matrices per edge triple alongside edge-type encodings. Substrate and node features are matched; the edge channel is not, since GAT-N-QoS consumes the scalar QoS aggregate $w(e)$ where HGT-QoS consumes all 16 dimensions (Section 6.2). Comparisons between the two therefore isolate relation-specific parameterization jointly with per-dimension QoS encoding, and we report the QoS factor separately in Section 7.3.1 rather than attributing the whole margin to typing. Two further factors are unmatched as published — parameter budget (434,620 against 28,168) and message-passing directionality — and Section 8.4 states what they leave open.
- **Training-Free Structural Baselines (Topo, Topo-QoS):** Topological baselines are evaluated on the derived Application–Library `DEPENDS_ON` projection (Section 3.2). This projected substrate is necessary because in raw publish–subscribe multigraphs, Application nodes never route messages directly, resulting in near-zero betweenness and bridge ratios that yield degenerate, uninformative scores.
- **Ground-Truth Independence Guarantee:** Crucially, **no predictor in either group accesses $G_{\text{structural}}$** : that raw topology is strictly reserved as the substrate for ground-truth simulation oracles (Section 4.4), a guarantee formally verified by `tests/test_independence_guarantee.py`.

Regardless of substrate, all variants are scored on an identical, independently resolved Application node set (V_{app} , Section 6.3).

6.3. Evaluation Metrics and Protocols

Figure accessibility. All figures in this paper use the high-contrast, colorblind-safe Okabe–Ito palette together with distinct marker, hatching, and node-shape encodings, so that every distinction carried by color is also carried by form and remains legible in monochrome.

- **Ranking Precision:** Evaluated via Spearman rank correlation (ρ) and Kendall’s rank correlation (τ) between predicted component rankings and ground-truth simulated impact $I^*(v)$ from the primary oracle (Section 4.3).
- **Critical-Set Identification:** Measured via $F_1@K$, Precision@ K , and Recall@ K for top- K critical components, where $K = \text{round}(0.20 \cdot |V_{\text{app}}|)$. Because predicted and ground-truth sets both contain exactly K elements, Precision, Recall, and F_1 coincide identically as the top- K set overlap.
- **Statistical Significance:** Assessed through paired Wilcoxon signed-rank tests [81] ($p < 0.05$) and non-parametric bootstrap 95% confidence intervals ($B = 2,000$) over folds [82, 83]. In the 12-fold LOSO design, power floor is $p = 0.00049$. Applying Holm’s step-down correction across the ten full-population rank contrasts in Sections 7.1–7.3.1, three survive: Topo-QoS over Topo ($p = 0.0010$), Topo over RM ($p = 0.0005$), and unweighted typing HGT over GAT-N ($p = 0.0010$), while QoS-weighted typing ($p = 0.0122$) and QoS edge ablation ($p = 0.0093$) remain nominally significant with 11/12 directional fold consistency.

Pre-registration. The primary out-of-distribution contrast (HGT-QoS vs. Topo-QoS under LOSO, 5 fixed seeds, fold as unit of analysis) was pre-registered before results were obtained, committing to report measured outcomes regardless of significance. The registration is a versioned file in the replication package rather than an entry in an independent registry, so its timestamp is auditable through the repository history but is not third-party certified; we state the weaker claim it supports. As reported in Section 7.1, the margin did not clear statistical significance.

6.3.1. Evaluation Population

Every predictor within a given evaluation table is scored on an identical node population, resolved strictly from scenario topology and simulation ground truth — never from any model’s predictions. Unless otherwise noted, this population is the **Application** set (V_{app}). This aligns with the framework’s primary objective (forecasting application-layer cascading failures) and ensures a fair common denominator across both typed and untyped predictors. Pooling node types into a single global ranking conflates distinct base rates and impact distributions, shifting the resulting rank correlation outside the envelope of per-type correlations (Section 7.3). We therefore report stratified, single-population metrics throughout and explicitly identify any pooled figures.

6.3.2. Evaluation Protocols

- **In-Distribution Evaluation:** 60% train / 20% validation / 20% test node splits over five seeds {42, 123, 456, 789, 2024}. Splits are deterministic functions of node ID and seed, inducing partition and training noise reflected in reported standard deviations.
- **Inductive Leave-One-Scenario-Out (LOSO):** Models train on eleven scenarios and test zero-shot on the held-out twelfth across all 12 folds. Parity is strictly enforced: every learned variant receives identical training sets of $N - 1$ graphs, message-passing depth is fixed at three layers, and checkpoint selection uses an inner validation split within the primary graph under an identical early-stopping protocol (Section 8.4). The outer holdout scenario participates in no training or selection decisions.
- **Real-World Architectural Transfer:** Models trained on synthetic topologies evaluate zero-shot on five open-source distributed systems without fine-tuning.

7. Results and Empirical Analysis

This section presents empirical results for RQ1–RQ5 across the twelve-fold inductive benchmark and five authentic open-source distributed systems. Evaluated populations are strictly stratified on the Application service set (V_{app}) under the input–label independence guarantee (Section 4.4).

7.1. RQ1: Graph Learning vs. Structural Baselines

Table 5 presents in-distribution held-out Spearman rank correlation (ρ) against simulated cascade impact $I^*(v)$ across all twelve distributed architecture scenarios ($n = 12$).

7.1.1. Out-of-Distribution (LOSO) Generalization

In inductive Leave-One-Scenario-Out (LOSO) cross-validation, models are evaluated on their capacity to predict cascading criticality over completely unseen system topologies:

Twelve LOSO folds are reported, comprising the eleven synthetic evaluation scenarios and the ATM case study, all specified in Table 4 (Section 6.1). In each fold, one scenario is held out for zero-shot testing while the model is trained exclusively on the remaining eleven. All variants are evaluated on the identical Application node set per fold (Section 6.3); paired Wilcoxon tests are conducted across the twelve folds, where the smallest attainable two-sided p is 0.00049. Per-fold evaluated populations range from 26 to 300 Application nodes, so $K = \text{round}(0.20|V_{\text{app}}|)$ ranges from 5 to 60. On $F_1@K$, HGT-QoS beats Topo-QoS in 8 of 12 folds ($\Delta = +0.154$, $W = 12.0$, $p = 0.034$) but separates from untyped GAT-N-QoS in 8 of 12 without

Table 5: In-distribution held-out Spearman ρ against simulated cascade impact $I^*(v)$: mean over five seeds with bootstrap 95% CI in brackets; n = held-out Application count. Substrates differ and the comparison is confounded in-distribution: HGT/HGT-QoS consume the native typed multigraph, while GAT/GAT-QoS and the topological baselines consume the Application-Library DEPENDS_ON flow projection (Section 6.2). The typed-untyped contrast below therefore mixes typing with multi-entity visibility here; the substrate-matched comparison is the LOSO one in Table 6, where both architectures read the same graph. Each seed redraws the 60/20/20 split as well as the initialization. The paired significance tests over these scenarios are in Supplementary Section S10.

Scenario	n	Topo	Topo-QoS	GAT	GAT-QoS	HGT	HGT-QoS
ATM System	5	0.538	0.557	-0.393	-0.080	0.492	0.348
AV System	16	0.187	0.797	0.816	0.465	0.637	0.558
Enterprise	60	0.443	0.793	0.779	0.481	0.861	0.878
Financial Trading	12	0.387	0.512	0.565	0.666	0.693	0.730
Healthcare	10	0.291	0.399	0.725	0.575	0.575	0.607
Hub-and-Spoke	14	0.179	0.429	0.363	-0.156	0.421	0.476
Industrial SCADA	28	0.601	0.710	0.656	0.478	0.787	0.839
IoT Smart City	40	0.320	0.397	0.580	0.538	0.849	0.850
Logistics Fleet	22	0.511	0.652	0.746	0.780	0.796	0.815
Microservices	18	0.219	0.344	0.351	0.363	0.141	0.664
Real-Time Gaming	15	0.360	0.802	0.464	0.471	0.651	0.641
Telecom RAN	24	0.402	0.422	0.608	0.350	0.591	0.526
Mean	—	0.370	0.568	0.522	0.411	0.624	0.661

Table 6: Inductive LOSO evaluation, Application population, twelve folds. Learned variants share training set, substrate, depth, and selection rule (Section 6.3), differing in typing and edge channel, and also – as published – in parameter budget and message-passing directionality, which Table 8 controls for. Fold score = mean over five seeds; **Fold** σ = spread of the twelve fold means; **Seed** σ = median within-fold spread (zero by construction for deterministic baselines); CI = bootstrap over folds ($B = 2,000$).

Predictor / Reference	Mean LOSO ρ	95% CI	Fold σ	Seed σ	Critical-Set $F_1@K$	Requires Training
<i>Training-free structural baselines</i>						
Topo	0.349	[0.254, 0.452]	0.173	—	0.366	No
Topo-QoS	0.553	[0.443, 0.657]	0.192	—	0.388	No
<i>Learned predictors (shared native substrate, matched training set)</i>						
GAT-N	0.317	[0.254, 0.381]	0.111	0.298	0.328	Yes
GAT-N-QoS	0.604	[0.538, 0.665]	0.112	0.024	0.431	Yes
HGT	0.551	[0.474, 0.617]	0.124	0.114	0.427	Yes
HGT-QoS	0.638	[0.561, 0.710]	0.133	0.052	0.424	Yes
<i>Diagnostic reference — not a ranking model</i>						
RM / $Q(v)$	0.205	[0.092, 0.320]	0.195	—	0.322	No

reaching significance ($\Delta = +0.034$, $W = 25.0$, $p = 0.301$): critical-set identification distinguishes the typed learned model from the training-free baseline, but not from untyped learning.

Label-noise ceiling. These correlations are bounded by the reproducibility of the target they are scored against. Re-running the ground-truth oracle across the five seeds gives a test-retest rank correlation between 0.811 and 1.000 across the twelve folds (median 0.982; nine of twelve at or above 0.95), with Microservices the least reproducible at 0.811. HGT-QoS’s $\rho = 0.695$ therefore recovers roughly 71% of the attainable signal against the median ceiling, and no predictor in Table 6 can exceed the reproducibility of its own labels. Top- K critical sets are the noisier construct by a wide margin: their cross-seed Jaccard has a median of 0.847 and falls to 0.370 (Logistics Fleet), 0.500 (Industrial SCADA), and 0.500 (Telecom RAN). That instability is the main reason the $F_1@K$ margins are less stable than the ranking margins, and it bounds how much weight any single critical-set comparison can carry. Notably, Microservices is both the least reproducible fold and one of the two on which typed learning loses (Section 7.2.1) — part of that deficit may be label noise rather than model failure.

Figure 2 summarizes these results alongside critical-set identification and inter-oracle agreement.
Key Insights for RQ1:

1. **Typed learning is the best configuration, but not demonstrably better than the QoS baseline.** HGT-QoS leads all predictors out-of-distribution ($\rho = 0.638$). Against training-free *Topo-QoS* it is $+0.085$ (9/12, $W = 20.0$, $p = 0.151$, CI $[-0.029, +0.194]$), an interval that includes zero; un-augmented HGT is indistinguishable from the baseline outright (-0.002 , 3/12, $p = 0.470$). Neither pre-registered contrast reaches significance, and we report that as the answer rather than as a near miss.
2. **A QoS-weighted structural score is a genuinely strong baseline — and untyped learning is worse than it.** *Topo-QoS* reaches $\rho = 0.553$ zero-shot, beating unweighted *Topo* on all twelve folds ($+0.204$, $p = 0.0005$). More pointedly, the untyped, unweighted learned model *loses* to it decisively (GAT-N, -0.236 , 2/12, $p = 0.0024$): on this task a homogeneous graph network trained on eleven architectures does not reach what a closed-form centrality score achieves with no training at all. Any claim that graph learning is *required* must be made against this baseline.
3. **Critical-set identification does not favor the typed model.** On $F_1@K$, HGT-QoS scores 0.424 against GAT-N-QoS’s 0.431 and HGT’s 0.427 — a three-way tie within noise — while all three beat *Topo-QoS* (0.388). The margin over the training-free baseline is real; the margin over untyped learning is not, and an earlier version of this paper claimed the latter.
4. **Power is not the limiting factor.** At $n = 12$ the design tolerates four lost folds and still reaches $\alpha = 0.05$, provided the losses are smallest in magnitude. HGT-QoS’s are not: it loses Enterprise (-0.335) and Telecom RAN (-0.169) to *Topo-QoS* by the two largest margins in the set, which is what holds W at 20.0. Enlarging the corpus will not resolve this; the inversions must be understood instead (Section 7.2.1).
5. **The explanation layer is weakly predictive, not noise.** $RM/Q(v)$ reaches $\rho = 0.205$, losing to unweighted *Topo* on every fold (-0.144), so no ranking claim is made for it. Its interval $[0.092, 0.320]$ stays above zero and it supplies interpretable diagnostics without training (Section 5). It appears in Table 6 as a reference point, not a competitor.

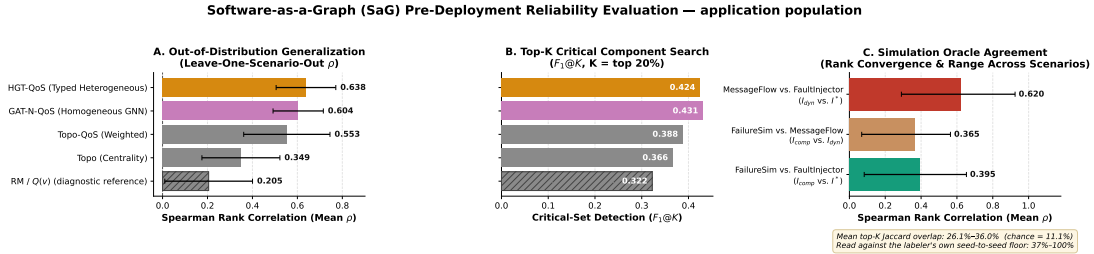


Figure 2: Results at a glance, Application population. (A) Out-of-distribution rank correlation per predictor across the twelve LOSO folds. (B) Critical-set identification at $K = 20\%$. (C) Pairwise rank agreement between the three simulation oracles, against the chance baseline. Panels A and B are read directly from the same artifact as Table 6 and panel C from that behind Supplementary Section S9; the ordering shown is whatever the data gives, which is why the untrained *Topo-QoS* baseline sits third rather than last.

7.1.2. The Active Stratum: Ranking Quality Separated From Inertness Detection

Every LOSO figure above is a correlation over the full held-out Application population, and between 21% (Microservices) and 52% (Healthcare) of that population carries exactly zero simulated impact depending on the fold. A predictor can therefore score well by separating components that can propagate a failure from those that cannot, without ordering the propagating ones correctly. Because these are different capabilities with different operational value, we re-score all twelve folds on the *active stratum* — the $n_{>0}$ components with strictly positive ground-truth impact — using the same predictions, folds, and seeds. Table 7 reports both.

Two consequences follow, and the first reverses a claim earlier versions of this paper made.

Table 7: LOSO means over twelve folds on the full Application population (ρ) and restricted to components with strictly positive ground-truth impact ($\rho_{>0}$). Same predictions, folds, and seeds; only the evaluated subset differs. Training-free baselines lose most of their apparent accuracy under the restriction; learned models lose far less.

Predictor	ρ (full)	$\rho_{>0}$ (active)	Retained
RM / $Q(v)$	0.205	0.102	49%
Topo	0.349	0.181	52%
Topo-QoS	0.553	0.280	51%
GAT-N	0.317	0.159	50%
GAT-N-QoS	0.604	0.328	54%
HGT	0.551	0.299	54%
HGT-QoS	0.638	0.356	56%

1. **The restriction costs every predictor roughly half its correlation, learned or not.** Retained fractions run from 49% (RM) to 56% (HGT-QoS) with no systematic separation between the training-free and the learned families — Topo-QoS retains 51%, GAT-N 50%, HGT 54%. An earlier version of this section reported that structural heuristics retain only 26–32% against 58–65% for learned models, and read that gap as the clearest evidence that learning contributes something a heuristic does not. On the present artifact that gap does not exist, and the claim is withdrawn. What the restriction shows is that roughly half of *everyone’s* full-population correlation is the separation of inert components from active ones, which is a property of the label distribution rather than a discriminator between methods.
2. **The ordering of methods is unchanged, and so are the verdicts.** On the active stratum HGT-QoS still leads ($\rho_{>0} = 0.356$), still ahead of Topo-QoS (0.280) by a margin in the same direction and of the same modest size as on the full population, and GAT-N-QoS (0.328) remains within reach of it. Nothing in Section 7.1 or Section 7.2 turns on whether the zero-impact components are included; we report both columns because they answer different operational questions, not because they disagree.

7.2. RQ2: Value of Typed Heterogeneity

RQ2 asks whether relation typing improves prediction over homogeneous message passing. Answering it requires ablating typing against a matched comparator, and the answer depends entirely on which comparator is chosen — which is the finding of this section.

- **Against an untyped, unweighted baseline, typing is decisive.** Under LOSO, where both architectures consume the identical native multigraph, HGT leads GAT-N by $\Delta\rho = +0.234$ ($\rho = 0.551$ vs. 0.317), winning *all twelve* folds ($W = 0.0$, $p = 0.0005$, Holm-corrected $p = 0.0020$, CI $[+0.170, +0.298]$). This is the largest and most consistent effect in the study, and it is the attainable floor of the signed-rank test at $n = 12$.
- **Once the QoS edge channel is present, typing adds almost nothing.** HGT-QoS leads GAT-N-QoS by $+0.035$ ($\rho = 0.638$ vs. 0.604), 9 of 12 folds, $W = 19.0$, $p = 0.1294$, Holm-corrected $p = 0.2588$. The interval spans zero. An earlier version of this manuscript reported this contrast as $+0.114$ at $p = 0.0122$ from a superseded artifact; that figure does not reproduce and is withdrawn.
- **In-distribution fitting (Table 5) is not evidence either way.** The homogeneous pair there reads the Application–Library projection while the typed pair reads the native multigraph, so that margin confounds typed message passing with multi-entity visibility. We report it as a fitting result only.

Typing and QoS encoding are substitutes, not complements. The two bullets above are the same factor measured against two different comparators, and Table 8 shows the symmetric result holds for the other factor: the QoS edge encoding is worth $+0.287$ over GAT-N when typing is absent ($p = 0.0010$, Holm 0.0029) and $+0.087$ over HGT when typing is present ($p = 0.2036$). Each mechanism, alone, lifts the plain baseline from $\rho = 0.317$ to roughly 0.55–0.60. Together they reach 0.638 — barely more than either achieves by itself. The interaction is sub-additive to the point of near-redundancy.

We read this as evidence that both mechanisms supply the model with the same underlying information: which relation a message crosses. Relation-specific parameters encode it in the weight matrices; the QoS edge vector encodes it in the edge features, since a topic’s declared contract co-varies with the kind of channel it is. Section 4.3 gives the corroborating measurement from the label side: $I^*(v)$ ’s ordering is 96.5% recoverable with no QoS term in the labeler at all, so neither channel can be tracking QoS-driven impact the oracle does not itself express. A corpus whose ground truth expressed deadline misses, durability replay or priority inversion would separate the two mechanisms far better than ours can, and we expect that to be the more informative experiment.

The practical consequence is a design recommendation rather than an architectural claim: on a target of this kind, a practitioner should adopt one of the two mechanisms and not expect the second to pay for itself. Which one is the cheaper question — the untyped QoS-weighted model (GAT-N-QoS) reaches $\rho = 0.604$ at 28,168 parameters against HGT-QoS’s 0.638 at 434,620, a $15.4\times$ capacity difference for $+0.035$ that does not clear significance.

All learned predictors in Table 6 run under strict substrate and training-set parity: every model receives all $N - 1$ training graphs, message-passing depth is fixed at three layers, and checkpoint selection follows the same rule — a validation split within the primary training graph (Section 6.3) — for every variant alike. The contrasts in Table 8 therefore isolate typing and the edge channel from substrate, training set, depth and selection rule. They do not isolate them from parameter budget or message-passing directionality, which remain unmatched; Section 8.4 states what that leaves open.

Table 8: Contrasts among the four reported learned arms, Holm-corrected within this family. These are measured results on the twelve LOSO folds, not confound controls; the arms that would isolate parameter budget, edge-channel width and directionality were not run (Section 8.4). The first two rows are the comparisons RQ2 and RQ3 headline; the second two are the same factors evaluated against the plain untyped, unweighted baseline. Parameter counts are measured on the LOSO primary training graph, whose relation set determines HGTConv’s size. Bootstrap CI is over the twelve fold deltas ($B = 2,000$).

Contrast	Factor varied	$\Delta\rho$	Won	W	p	p_{Holm}	Verdict
HGT-QoS vs. GAT-N-QoS	Typing, QoS present	+0.035	9/12	19.0	0.1294	0.2588	Not significant
HGT-QoS vs. HGT	QoS edge, typing present	+0.087	10/12	22.0	0.2036	0.2588	Not significant
HGT vs. GAT-N	Typing, QoS absent	+0.234	12/12	0.0	0.0005	0.0020	Significant
GAT-N-QoS vs. GAT-N	QoS edge, typing absent	+0.287	11/12	1.0	0.0010	0.0029	Significant

7.2.1. Where Typed Learning Fails, and How It Can Be Detected

HGT-QoS loses to Topo-QoS on three folds, but only two of them are substantive: Enterprise ($\rho = 0.569$ vs. 0.838) and Microservices (0.483 vs. 0.563). The third, Real-Time Gaming (0.776 vs. 0.780), is a tie to within 0.004 and carries no interpretive weight. It is the two substantive inversions that hold the RQ1 comparison below significance, and both are folds on which the training-free baseline is unusually strong, which suggests the model is discarding structural signal the baseline retains rather than failing to learn.

Absence of a Label-Free Confidence Signature. We evaluated whether the standard deviation of predicted scores $\hat{\sigma}$ over held-out applications could signal model reliability at inference time without labels. Although the two worst-performing folds (Enterprise, $\hat{\sigma} = 0.111$; Microservices, 0.138) exhibit low dispersion, the correlation does not hold across the full benchmark: $\hat{\sigma}$ correlates with the margin over Topo-QoS at $\rho_s = -0.126$ ($p = 0.697$) for HGT-QoS, and the second-lowest dispersion fold (Healthcare, 0.123) yields one of the largest positive margins ($+0.284$). Neither graph size (rank correlation with margin -0.357 , $p = 0.255$) nor edge density reliably flags fold difficulty in advance.

Feature scale drift across scenarios (documented in `results/feature_shift_diagnostic.md`) remains the primary explanation for the Enterprise deficit, as Enterprise is the largest graph (520 nodes) where feature scaling disparities are most acute. Because model hyperparameters are strictly held constant across folds by protocol, a practitioner currently has no automated label-free signal to pre-determine whether an unseen architecture will favor the learned model or the training-free baseline (Section 8.4).

7.3. RQ3: Ablations and Sensitivity Analysis

This section reports the ablations that bear on a headline claim — the QoS edge encoding, cross-oracle agreement, and the per-type stratification that governs how every other result in this paper is read. The parameter-sensitivity sweeps over the explanation layer’s ten declared weight constants establish robustness rather than any finding of their own, and are reported in full in the supplementary material (Supplementary Sections S1–S2). Their collective result is stated here so the body remains self-contained: of the ten constants, only the Fault-Tolerance/Availability blend r_α and the AHP shrinkage λ carry appreciable influence on ρ ($\mu^* = 0.144$ and 0.117 under Morris screening, against ≤ 0.023 for the remaining eight), and no setting of the topic-weight or QoS sub-weight constants would change any comparison reported above. The elicited AHP weights are, notably, *anti*-predictive: rank correlation falls monotonically from 0.319 under a uniform prior to 0.200 under raw AHP judgment. We retain them because RM is an attribution instrument rather than a ranking model, and discuss that trade in Section 8.4.

7.3.1. QoS Feature Ablation

To isolate the specific empirical contribution of the continuous-categorical QoS edge features (Section 4.1.1), we evaluated **HGT**, an un-augmented ablation of HGT-QoS whose edge features contain only scalar coupling and relation one-hot encodings.

Under the inductive LOSO evaluation, the QoS edge encoding’s value depends entirely on whether relation typing is already present — the mirror image of the typing result in Section 7.2, and the second half of the same finding.

Without typing, the encoding is decisive. GAT-N-QoS reaches $\rho = 0.604$ against GAT-N’s 0.317: $\Delta\rho = +0.287$, won in 11 of 12 folds, $W = 1.0$, $p = 0.0010$, Holm-corrected $p = 0.0029$, CI $[+0.207, +0.365]$. *With typing, it is not significant.* HGT-QoS reaches 0.638 against HGT’s 0.551: $+0.087$, 10 of 12 folds, $W = 22.0$, $p = 0.2036$, Holm-corrected $p = 0.2588$. An earlier version of this manuscript reported the typed gain as $+0.054$ at $p = 0.0093$ from a superseded artifact and treated it as an independent contribution on top of typing; the effect does not reproduce at that significance and the independence claim is withdrawn.

The encodings also improve optimization reproducibility, and here the asymmetry runs the other way. The median within-fold standard deviation across five seeds is 0.024 for GAT-N-QoS against 0.298 for GAT-N — more than a tenfold reduction — and 0.052 for HGT-QoS against 0.114 for HGT. An untyped model without the QoS channel is the least stable configuration in the study by a wide margin, and either mechanism stabilises it. This is consistent with the ranking result: both channels tell the model which relation an edge belongs to, and a model given neither is left to infer it from topology alone.

The target itself is nearly QoS-free, which bounds what either channel can be crediting. The gains above are earned against $I^*(v)$, whose ordering is 96.5% recoverable with no QoS term in the labeler at all (mean $\rho = 0.965$ between the shipped ladder and a topology-only relabeling of the same twelve folds; Section 4.3). The QoS edge channel therefore cannot be helping the model track QoS-driven impact that the oracle does not itself express, which is the most direct evidence we have for reading it as a relation-identity channel rather than a contract-semantics one. Where the label *does* move under QoS is its top- K boundary (Jaccard 0.678 against the topology-only arm) rather than its ranking. A corpus whose oracle expressed QoS-driven impact in its ordering — deadline misses, durability replay, priority inversion under load, none of which I^* observes — would be a stronger test of the encodings than the one we report.

QoS Parameter Variance. Modal QoS shares range from 29% to 89% across the twelve scenarios (Supplementary Section S5), ensuring that every fold carries genuine variation in declared reliability, durability, and priority. As noted in Section 4.1.1, one schema dimension (`max_blocking_ms_log`) remains zero throughout the corpus as a reserved extension point, while the declared deadline populates the other two (`has_deadline`, `deadline_ns_log`) on 75% of topics; reported gains therefore stem from six active dimensions.

7.3.2. Convergent Validity Over Simulation Oracles

The three reliability-facing oracles measure distinct constructs, so we checked whether they agree before treating any one of them as ground truth. Over the twelve inductive folds on the Application population, the

behavioural queue-flow oracle and the topological cascade injector agree at mean Spearman $\rho = 0.620$ (top- K Jaccard 0.365 against 0.111 expected by chance), against I^* 's own seed-to-seed test-retest of 0.811–1.000. The agreement is therefore substantial but distinctly below label noise, which is the reading we want: an oracle reproducing another to within its own reproducibility would be re-measuring the same topology rather than corroborating it. Two boundaries qualify this — a large share of the agreement is the two oracles concurring on which components are *harmless*, and I_{dyn} has a measured noise floor of its own that the headline does not correct for. Supplementary Section S9 reports the full pairwise table, the zero-excluded correlations, and the multi-seed floors for both non-primary oracles.

7.3.3. Node-Type Stratification and Attention

One result governs how every other number in this paper is read. Measured against $I_{\text{comp}}(v)$ over the eight scenarios of the detection benchmark, stratified RM rank correlations are $\rho = 0.566$ (Application), 0.119 (Broker) and 0.244 (Node), while pooling all types collapses the correlation to $\rho = 0.098$ — below every per-type value it aggregates, which is Simpson's paradox in its textbook form. This is why every evaluation here is reported on a single stratum, and why pooled critical-set figures should be read as inflated wherever they appear. Supplementary Section S6 reports the rule-based anti-pattern catalog evaluated on the same benchmark; its summary is that the catalog flags 93.8% of scored components and therefore does not discriminate, so critical-set identification is delegated to the continuous rankers of Sections 7.1–7.2.

Aggregated by relation type over the ATM case study, first-layer mean HGT attention orders **USES** into libraries (0.227) above publish-subscribe channels (0.163–0.176), but the spread across all eight relation types is narrow (0.15–0.23) and driven substantially by destination in-degree artifacts. Supplementary Section S8 gives the layer-wise distribution and heatmap, and confirms that typed attention remains active across relation types without establishing a statistically distinct ordering.

7.4. RQ4: Real-World Distributed Architecture Validation

We evaluated the framework on five open-source distributed systems transcribed from public repositories: Online Boutique, Train-Ticket, Home Assistant, Autoware.universe (ROS 2), and EdgeX Foundry. All carry labels from the same simulation oracles used throughout, testing topological transfer rather than agreement with field failures.

We conduct two distinct real-world evaluations. First, the closed-form explanation layer $Q(v)$ is evaluated against $I_{\text{comp}}(v)$ in Supplementary Section S7, achieving strong correlation across all five systems ($\rho = 0.514$ –0.800) and outperforming degree centrality. Both training-free references are scored here. **Topo-QoS** was absent from earlier versions of Table 9 on the stated grounds that the open-source adapters carried no QoS contracts; that was a defect in how its betweenness was projected rather than a property of the data, and Section 8.4 records the correction.

7.4.1. Zero-Shot Transfer of the Learned Model

To test generalization to architectures outside our generator, we trained HGT-QoS on all twelve synthetic scenarios and evaluated it zero-shot across the five open-source systems against $I^*(v)$ (five seeds). To mitigate cross-scenario feature-scale drift, this transfer evaluation applies within-graph rank normalization, 2 message-passing layers, and 150 epochs. No real-world system contributed training gradients or was used for checkpoint selection. Table 9 presents the resulting transfer performance.

Key Insights for Real-World Transfer:

1. **The full-population figure is not a ranking result.** On the whole Application population HGT-QoS reaches $\rho = 0.767$ against 0.511 for Topo, 0.526 for Topo-QoS and 0.516 for RM, leading on four of five systems. Read alone that looks like successful zero-shot transfer. It is not, because between 4% (Home Assistant) and 46% (Train-Ticket) of Applications in these architectures carry exactly zero simulated impact, and a correlation over a population heavily tied at zero rewards separating the inert from the active at least as much as ordering the active correctly.

Table 9: Zero-shot transfer to five open-source systems, scored against $I^*(v)$ on the Application population. HGT-QoS trains on all twelve synthetic scenarios ($\pm$ = spread over five seeds); RM and Topo are training-free, scored on identical labels and nodes. $\rho_{>0}$ restricts the correlation to the $n_{>0}$ components that actually propagate a failure and is the column to read for ranking quality; full-population ρ conflates that with separating active from inert. Both training-free references are scored on the same labels and nodes (Section 8.4).

Real-World Architecture	$ V_{\text{app}} $	RM ρ	Topo ρ	Topo-QoS ρ	HGT-QoS ρ	HGT-QoS $\rho_{>0}$	$n_{>0}$	$F_1@K$
Cloud Microservices Mesh	22	0.777	0.891	0.888	0.649 ± 0.131	-0.029	18	0.400
Train-Ticket Booking Mesh	41	0.713	0.528	0.541	0.776 ± 0.004	-0.213	22	0.450
Autoware.universe (ROS 2)	32	0.357	0.307	0.378	0.734 ± 0.054	+0.559	28	0.633
EdgeX Foundry (Industrial IoT)	22	0.470	0.534	0.534	0.804 ± 0.040	+0.304	19	0.500
Home Assistant (Smart Home)	24	0.265	0.297	0.289	0.872 ± 0.035	+0.704	23	0.600
Mean	—	0.516	0.511	0.526	0.767	+0.265	—	0.517

2. **Restricted to components that actually propagate failures, transfer is not established.** On the active stratum the mean falls from 0.767 to +0.265, and two of the five systems invert: Cloud Microservices to $\rho_{>0} = -0.029$ ($n = 18$) and Train-Ticket to -0.213 ($n = 22$). Both are the microservice call-tree architectures. The three pub-sub systems hold up (+0.559 Autoware, +0.704 Home Assistant, +0.304 EdgeX). **We therefore report RQ4 as a negative result:** learned relational transfer to authentic open-source architectures is not established by this evidence. Section 8.3 states why the architectural explanation for the split is a conjecture rather than a tested hypothesis.
3. **The QoS-weighted baseline is now scored, and it sharpens one comparison.** Topo-QoS reaches $\rho = 0.888$ on Cloud Microservices — the system on which the learned model does worst (0.649) and inverts on the active stratum. Earlier versions of this paper could not report that column and compared against unweighted Topo instead; the corrected comparison makes the learned model’s deficit on synchronous call trees sharper, not milder.
4. **What the full-population number does support.** Separating components that propagate failures from those that do not is the operationally useful half of the task — a gate that correctly identifies which components cannot cause a cascade has narrowed the review surface, even if it orders the remainder poorly. The $F_1@K$ column (0.517 mean, and 0.633 on Autoware) is the honest expression of that capability, and it is what a practitioner would act on.

7.5. RQ5: Analysis Cost and Its Comparison Against Simulation

RQ5 quantifies computational overhead and sustainability during CI/CD evaluation, with per-stage latencies reported in Table 10:

Table 10: Per-stage latency of the inference pipeline across scaling graph sizes (CPU, median of 3 runs).

$ V $	$ E $	Analyze (s)	Graph→tensor (s)	HGT forward (ms)	Analyze : forward
249	1,127	1.74	0.010	26.5	66×
499	2,402	8.32	0.022	16.4	509×
999	6,422	44.54	0.056	21.1	2,108×
1,998	19,301	239.34	0.157	56.2	4,259×

The neural model is the cheapest stage, and the deterministic one is expensive. At 2,000 components the HGT forward pass takes 56 ms against 239 s for deterministic structural analysis — a ratio of 4,259×. That ratio locates cost *inside* the pipeline; it is not the cost of evaluating an architecture. Indices 0–17 of every node feature vector (Section 3.4) — betweenness, closeness, reverse PageRank, articulation and bridge scores — are products of that same analysis stage, so the forward pass cannot run without it. End-to-end evaluation of an unseen 2,000-component architecture costs about four minutes, of which the learned model is 0.02%; the 56 ms figure is the marginal cost of re-scoring an already-analyzed graph. Across the eight-scenario detection benchmark the complete gate (structural analysis plus 18 anti-pattern detectors) runs in 0.04–82.7 s, the upper bound being the 520-component Enterprise mesh.

Cost is dominated by one metric, and it grew. Measured cost now tracks the stage’s $O(|V|^2 + |V||E|)$ bound closely: from 249 to 1,998 components wall-clock rises 138× against a 137× growth in $|V||E|$. The

dominant term is the Connectivity Degradation Index, which is computed for every node in the main connected component rather than for articulation points alone. That choice is deliberate and is a correctness requirement rather than an oversight: gating CDI to articulation points leaves it identically zero for every node whose removal does not literally disconnect the graph, which drives $A(v)$ to a near-constant in the redundant multi-publisher topologies this framework targets. The cost is the price of a non-degenerate Availability score, and we report it rather than the cheaper gated variant we could have measured.

7.5.1. The Gate Is Not Cheaper Than the Simulation It Replaces

The framing that motivated this analysis — static gating as a low-cost substitute for dynamic simulation — does not survive measurement against our own oracle. Timing the **FaultInjector** labeling sweep (five seeds, node types Application/Broker/Library, the full ground-truth run) on the same corpus and the same idle hardware gives 0.14–7.2 s per scenario, against 0.04–82.7 s for the analysis gate. Both maxima belong to the 520-component Enterprise mesh, so the largest scenario compares 7.2 s of simulation against 82.7 s of static analysis: **the gate costs roughly eleven times more than the simulation it is meant to displace.**

This finding refutes the assumption that static analysis is computationally cheaper than in-process simulation: breadth-first cascade traversal is simpler than computing all-pairs connectivity degradation. However, the operational distinction is what the two require: dynamic simulation and chaos engineering demand provisioned clusters and runtime execution, whereas SaG runs directly on manifests during CI/CD. The measurements support the claim that analyzing hundreds of components fits within pull-request budgets without runtime infrastructure, but they do not support general computational superiority over simulation.

8. Discussion, Threats to Validity, and Limitations

8.1. Discussion and Practical Implications

When to use Topo-QoS, and when to use HGT-QoS. The results do not support a simple recommendation of the learned model over the closed-form one, and we set out the trade-off as measured rather than as hoped.

1. **Training-free ranking (Topo-QoS).** It requires no training, no checkpoint storage, and no retraining as a corpus evolves, reaching $\rho = 0.553$ zero-shot across twelve unseen synthetic architectures and 0.526 across five open-source systems. Nothing in this study establishes that a learned model beats it on ranking: the margin is +0.085 ($p = 0.151$) with an interval spanning zero, and on the one real system where the two diverge sharply it is the heuristic that wins (0.888 vs. 0.649 on Cloud Microservices). For a team that wants a criticality ordering and nothing more, this is the defensible default, and we say so despite proposing the alternative.
2. **One learned mechanism, not two (GAT-N-QoS or HGT).** The clearest learned result is that either relation typing or a QoS edge channel lifts an untyped, unweighted model substantially — +0.234 and +0.287 respectively, both surviving Holm correction — while adding the second to the first does not. A practitioner should therefore pick one. On cost grounds the untyped QoS-weighted model is the better pick: $\rho = 0.604$ at 28,168 parameters against HGT-QoS’s 0.638 at 434,620. Note also that the untyped, *unweighted* model (GAT-N, $\rho = 0.317$) is worse than the training-free heuristic, so “use a GNN” is not by itself sound advice here; which relational signal it is given decides whether it is worth training at all.
3. **Capabilities without a closed-form counterpart.** Two remain, and neither is evaluated here. Typed relational attention exposes *which* channels mediate a cascade rather than only which components rank highly (Supplementary Section S8 illustrates this on one topology and is explicitly not evidence of a general effect). Relationship-level criticality (I_{edge} , Eq. (12)) scores individual dependencies rather than components, which is what circuit-breaker or bulkhead placement actually requires and which a node ranking cannot express; this paper defines the edge oracle and the model’s edge head but presents no evaluation of one against the other, so the capability is available and untested. Where the extra machinery is affordable these are the substantive reasons to prefer a typed model — as design arguments, not as measured advantages.

A gate we can no longer recommend. An earlier version of this work proposed a tiered gate: run the learned model by default and fall back to **Topo-QoS** when the model’s prediction dispersion $\hat{\sigma}$ fell below a threshold. That heuristic does not replicate: across twelve folds, $\hat{\sigma}$ correlates with the margin over **Topo-QoS** at $\rho_s = -0.126$ for HGT-QoS (the wrong sign, Section 7.2.1). We therefore withdraw the automated fallback recommendation. Instead, because both engines run in seconds, our results support running both concurrently and escalating ranking disagreements to human architectural review.

Dual-Engine Consensus Protocol. Because both engines run in seconds, SaG ships a dual mode (**saag-predict** with **--predictor-mode dual**) that scores each manifest with **HGT-QoS** and **Topo-QoS** together and partitions the result: components in the top- K set of both are unanimous risks worth prioritising, while components the two rank far apart are escalated to human architectural review rather than resolved automatically. The design follows from the measurement — we have no label-free signal that says which engine to trust on an unseen architecture, so the honest response is to surface the disagreement instead of hiding it behind a threshold. We report the protocol as a deployment recommendation; its triage value is not evaluated here.

Role of the Explanation Layer. The RM attribution profile ($Q(v)$, Section 5) provides transparent, standards-compliant architectural diagnostics aligned with ISO/IEC 25010. By separating single-point-of-failure exposure (Availability) from wide error propagation reach (Fault Tolerance), RM provides qualitative remediation guidance (e.g., distinguishing whether a component requires replication or decoupling) that purely numeric rankers and simulation oracles cannot provide.

8.2. Performance and Computational Sustainability Implications

What sustainability means for a pre-deployment gate. Green software engineering distinguishes the energy a system consumes from the energy its *development and assurance* consume [29], and the machine-learning literature has concentrated on the second [84, 85, 86, 87, 30, 31]. Reliability assurance sits in the same category and is rarely measured: chaos engineering, hardware-in-the-loop benches and staging-cluster fault injection consume cluster-hours per sweep, on every pull request that triggers them. Two findings bear on this, at the strength the measurements support.

The first is unambiguous. *Within* the pipeline the neural model is negligible: a 56 ms forward pass on a 2,000-component system against 239 s for deterministic structural feature extraction, a ratio of $4,259\times$ (Section 7.5). Whatever pre-deployment dependability analysis costs, adding a graph neural network is not what makes it expensive — which, for a special issue asking whether AI techniques can be afforded in a sustainability-conscious process, is the directly relevant result.

The second is a difference in kind rather than degree, and it is weaker than it first appears. What manifest-time analysis eliminates is *infrastructure*: nothing must be deployed, kept warm, or torn down, and no fault is injected into anything a user could be holding. But that argument does not distinguish SaG from its own oracle, which also reads a manifest and costs eleven times less (Section 7.5.1); it distinguishes static analysis in general from chaos engineering on a provisioned cluster, and we have not measured a chaos-engineering baseline, so its magnitude is an assertion rather than a result.

The efficiency claim we withdraw, and where the cost actually sits. Static analysis does not reduce computation relative to in-process simulation: global connectivity degradation (82.7 s on the Enterprise mesh) is roughly eleven times slower than BFS cascade traversal (7.2 s). We withdraw any general claim of computational efficiency relative to simulation. The expense is concentrated in one metric and is a deliberate accuracy purchase: the Connectivity Degradation Index is computed for every node in the main component rather than for articulation points alone, and gating it would restore roughly an order of magnitude of speed at the price of a degenerate Availability score. A deployment needing the speed more than the single-point-of-failure sensitivity could make the opposite trade. This also locates where optimization effort belongs: the sustainability of this framework is a question about one graph-theoretic routine, not about its use of machine learning. Settling it comprehensively requires energy counters rather than wall-clock time [30]; and while preventing cascading failures plausibly reduces datacenter compute lost to retry storms and restart loops, we treat that as motivation rather than an empirical claim.

8.3. Threats to Validity

Construct Validity. Our primary ground-truth impact oracle $I^*(v)$ is derived from discrete-event cascade simulation on structural models rather than live outages. Evaluating construct divergence against the queue-flow discrete-event simulator $I_{\text{dyn}}(v)$ and composite oracle $I_{\text{comp}}(v)$ (Supplementary Section S9) reveals substantial but sub-ceiling correlation with I_{dyn} ($\rho = 0.620$ against a 0.811–1.000 label test–retest ceiling), which traverses related but not identical failure semantics. However, agreement with I_{comp} is moderate ($\rho = 0.395$), and top- K critical-set Jaccard reaches only 0.27–0.37 due to non-linear cascade threshold sensitivity and tied zero-inflation (Supplementary Section S2). Thus, convergent validity across simulation paradigms holds for ranking but is weaker for critical-set boundaries; no oracle is validated against live outages (Section 4.4). For Maintainability, $I_M(v)$ traverses the same dependency topology from which $M(v)$ is scored; independent churn validation remains future work.

Internal Validity. Potential feature leakage is prevented by strict graph view separation: predictors operate exclusively on G_{analysis} , whereas ground-truth simulation oracles operate on $G_{\text{structural}}$, formally asserted in continuous integration. Substrate parity is rigorously maintained: learned models (HGT-QoS, GAT-N-QoS) share identical training sets, depths, and early-stopping rule—a validation split within the primary training graph (§6.3), applied uniformly across variants. In the QoS schema, four dimensions (reliability, durability, transport priority, heterogeneity flag) capture active operational middleware configurations, and two further dimensions encode the declared deadline, which is populated for 75% of topics across the twelve evaluation scenarios. The remaining dimension (max blocking) is a reserved extension point that is zero throughout the corpus.

External Validity. Our evaluation spans twelve synthetic architectures and five open-source systems. Zero-shot transfer to real systems is *not* demonstrated on active components (mean $\rho_{>0} = +0.265$, inverting on the two microservice call trees, Section 7.4.1). The explanation we offer for that pattern — that pub-sub meshes and synchronous RPC call trees propagate failure in opposite directions along the edges the model learned — was arrived at after seeing which systems failed, on a sample of five. It is a conjecture consistent with the data, not a tested hypothesis, and three systems against two cannot support a statistical claim either way. We state it because it is actionable (it predicts which architectures the current corpus cannot prepare a model for) and label it as post-hoc for the same reason. Furthermore, while synthetic QoS profiles exhibit genuine variance (modal shares 29–89%, Section 7.3.1), their alignment with production distributions remains unverified. Finally, our timing evaluations scale to 2,000 components; architectures an order of magnitude larger cannot be assumed to fit PR budgets given the $O(|V|^2 + |V||E|)$ dominant stage.

Conclusion Validity. Given heavy-tailed impact distributions, statistical analyses use non-parametric rank correlation (Spearman ρ , Kendall τ), bootstrap confidence intervals ($B = 2,000$), and paired Wilcoxon signed-rank tests, with the fold or scenario as the unit of analysis. Two hazards recur and shape how we report. Pooling across heterogeneous entity types triggers Simpson’s paradox — pooled $\rho = 0.098$ sits below every per-type value it aggregates (0.119–0.566) — so all headline figures are stratified on a single population. And rank correlation over zero-inflated labels conflates ordering the active components with separating them from inert ones, which is why we report zero-excluded correlations alongside full-population ones wherever the label distribution permits. Where the two disagree, as on the real-world systems, we read the zero-excluded figure as the ranking result.

Two further limits bound every p -value and interval we report, and neither is removable by analysis. The folds are not independent replicates: any two LOSO models share ten of their eleven training graphs, so a signed-rank test over folds treats as independent observations that are strongly coupled, and there is no unbiased estimator of the variance of a cross-validation estimate under this design. Our intervals and p -values are therefore optimistic by an amount we cannot quantify, and we report them as the conventional summary rather than as calibrated ones. Separately, the twelve synthetic topologies are draws from a single parameterized generator sharing fan-out, colocation and QoS-categorical distributions (Supplementary Section S5), so “generalizes to unseen architectures” means, strictly, unseen draws from one generator. The five open-source systems are the only architectures in this study whose structure we did not specify, and they are the ones on which transfer is not established.

8.4. Limitations and Future Work

A baseline that was missing for the wrong reason. Earlier versions of this work omitted Topo-QoS from Table 9, reporting that the open-source adapters carried no QoS contracts to weight with. That was incorrect. Every topic in all five adapters declares durability, reliability and transport priority, and projecting $w(t)$ onto the structural edges yields non-unit weights on roughly half of them; the obstacle was a defect in the projection guard, which computed QoS-weighted betweenness on the raw multigraph — where Application nodes never route messages and the score is degenerate by construction — rather than on the `DEPENDS_ON` projection every other topological baseline uses. With that corrected the column is scorable, and Table 9 reports it. We record the error rather than quietly filling the gap, because for two revisions it meant the real-world comparison was made against the weaker of the two training-free references while the stronger one was described as unavailable.

The explanation layer is not validated as an explanation. SaG separates Availability from Fault Tolerance on the hypothesis of distinct repairs, but we do not evaluate whether practitioners find this actionable. Furthermore, elicited AHP weights perform worse than a uniform prior at ranking (Section 7.3); counterfactual mutation tests and user studies are needed for validation.

Two confounds in the typing result remain uncontrolled. The contrasts of Table 8 hold substrate, training set, depth and selection rule constant, but two factors separating the typed and untyped architectures are unmatched as published. *Parameter budget:* on the relation set of the LOSO primary training graph, HGT-QoS carries 434,620 parameters against GAT-N-QoS’s 28,168. *Directionality:* HGT applies a reverse-direction `HGTConv` (103,725 parameters) so a node sees its upstream neighbourhood, whereas the homogeneous baseline propagates along native edge direction only — which matters because $I^*(v)$ is a downstream-reachability functional, so a model able to look upstream has an advantage on this target unrelated to typing. The arms that would isolate each (a capacity-matched homogeneous baseline, one reading the full sixteen-dimensional edge channel, and a forward-only heterogeneous model) are implemented and registered in the replication package but were not run at the reported budget, so no value is reported for them. The consequence is specific and we do not minimise it: the surviving +0.234 is consistent with relational typing acting as an inductive bias, and equally consistent with a capacity or directionality advantage. Running those three arms is the single most informative extension to this study.

Model selection is made on the training distribution. Early stopping in LOSO uses an inner validation split within the primary training graph (Section 6.3). While a protocol limitation under distribution shift, the rule is uniform across variants and cannot manufacture the typed-versus-untyped margin. Held-out scenario validation (`--inner-val-scenario auto`) is our prioritized extension.

No label-free reliability signal. Prediction dispersion does not replicate as a fallback indicator on this corpus (Section 7.2.1); discovering a reliable OOD confidence signal remains an open problem.

Scale is bounded by the deterministic stage, not the model. Our timings reach 2,000 components. Beyond that the binding constraint is the $O(|V|^2 + |V||E|)$ feature-extraction stage rather than the network (Section 7.5), so the useful optimizations are graph-theoretic: caching structural metrics across pull requests and recomputing only over the k -hop neighbourhood a manifest change touches. Training, separately, would need mini-batch subgraph sampling (GraphSAINT [88] or HGT’s own layer-dependent importance sampling) to reach the $|V| > 10^5$ regime of cross-organizational service fleets, since full-batch message passing expands its receptive field exponentially with depth. Neither is a limitation this evaluation encountered; both are what a deployment at an order of magnitude more scale would encounter first.

Future Directions: Distributed AI, Power Testbeds, and Self-Healing. We envision three primary extensions: (1) modeling distributed LLM serving clusters (e.g., vLLM, DeepSpeed) to mitigate straggler-induced GPU dissipation; (2) measuring hardware energy directly via RAPL/NVML to compare static gating against live chaos sweeps in joules; and (3) advancing from predictive diagnostics to prescriptive synthesis, automatically generating pull requests with circuit breakers, broker replicas, and tuned QoS parameters to resolve single points of failure.

9. Conclusion

This work introduced **Software-as-a-Graph (SaG)**, a pre-deployment Static System Analysis framework that combines a relation-specific Heterogeneous Graph Transformer for failure-impact forecasting with an interpretable ISO/IEC 25010 Reliability–Maintainability attribution layer, both operating on a typed multigraph derived from Architecture-as-Code manifests with no runtime telemetry.

The central empirical finding is that the framework’s two architectural mechanisms are substitutes rather than complements, and that this is visible only when they are ablated separately. Against a plain untyped, unweighted graph model, relation typing is worth $\Delta\rho = +0.234$ under inductive distribution shift (12 of 12 folds, Holm-corrected $p = 0.002$) and the 16-D QoS edge encoding is worth $+0.287$ (11 of 12, $p = 0.003$). Either mechanism alone recovers most of what the full model achieves. Adding the second to the first does not: typing contributes $+0.035$ ($p = 0.129$) once the QoS channel is present, and the QoS channel $+0.087$ ($p = 0.204$) once typing is. We read this as evidence that both mechanisms encode largely the same relational information about this target, a reading the oracle supports directly — $I^*(v)$ ’s ordering is 96.5% recoverable with no QoS term in the labeler at all, so neither channel can be tracking QoS-driven impact the ground truth does not itself express.

We are equally clear about what is not established. Against an unparameterized QoS-weighted centrality score, learned ranking is not superior ($+0.085$, $p = 0.151$), and the untyped learned model is measurably *worse* than that heuristic (-0.236 , $p = 0.002$). On the five open-source systems, full-population correlation of $\rho = 0.767$ falls to $+0.265$ once restricted to components that actually propagate failures and inverts on both microservice call-tree architectures, so zero-shot transfer to authentic systems is a negative result on this evidence. The confound controls that would separate relational typing from parameter capacity and message-passing directionality are implemented but unrun, so the mechanism behind the surviving margins remains underdetermined. A label-free confidence signal we previously reported does not replicate. The explanation layer’s elicited AHP weights measurably worsen ranking relative to a uniform prior, and we have no independent evidence that they improve attribution.

What the work contributes is therefore a reproducible typed-multigraph formulation of pub-sub architecture, a corpus that regenerates byte-identically from committed configurations, a measurement of where two widely-assumed architectural inductive biases help and where they cease to compose, and a pre-deployment pipeline whose learned component is its cheapest stage by three orders of magnitude while its deterministic stage is eleven times more expensive than the simulation it was meant to replace. Whether such a pipeline predicts failures that actually occur — rather than failures a simulator produces — is the question we most want answered next, and it requires field data no static corpus can supply.

Declarations

CRedit Authorship Contribution Statement. **Ibrahim Onuralp Yigit:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Visualization. **Feza Buzluca:** Conceptualization, Methodology, Validation, Writing – review and editing, Supervision.

Declaration of Competing Interest. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Supplementary Material. Parameter sensitivity analyses (OFAT, Morris screening, threshold and normalization sweeps, zero-inflation diagnostics, and HGT attention distributions) are provided in the online supplementary document (Sections S1–S8).

Data Availability. The complete replication package—including synthetic datasets, generator configurations, simulation harnesses, real-world adapters, model checkpoints, and analysis scripts—is openly available on Zenodo under DOI 10.5281/zenodo.14922108 [89]. Synthetic datasets regenerate byte-identically from committed configurations. Reported table values are deterministically verified against backing JSON artifacts via the included `reproduce/reconcile_manuscript.py` script, which also refuses to report a clean run when

a declared artifact is absent or was produced from a modified working tree. Its coverage is the table cells and two named prose quantities, not every figure quoted in running text; we state the scope because a mechanical check is only as strong as what it reads.

Declaration of Generative AI and AI-Assisted Technologies. During the preparation of this work the authors used Anthropic’s Claude to assist with LaTeX typesetting and improve readability. After using this tool, the authors reviewed and edited the content and take full responsibility for the published article. No generative AI tool was used to design the study, analyze research data, or generate experimental results: all figures and tables are rendered deterministically from replication package artifacts.

References

- [1] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: Design, architecture, and uses in the wild, *Science Robotics* 7 (66) (2022) eabm6074.
- [2] J. Kreps, N. Narkhede, J. Rao, Kafka: A distributed messaging system for log processing, in: *Proc. 6th Int. Workshop on Networking Meets Databases (NetDB)*, 2011.
- [3] Object Management Group, Data distribution service (dds), Tech. Rep. formal/2015-04-10, version 1.4, Object Management Group (2015).
- [4] OASIS, MQTT version 5.0, OASIS Standard, <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html> (accessed 9 September 2026) (2019).
- [5] N. Dragoni, S. Giallorenzo, A. L. Lafuente, M. Mazzara, F. Montesi, R. Mustafin, L. Safina, Microservices: Yesterday, today, and tomorrow, in: *Present and Ulterior Software Engineering*, Springer, 2017, pp. 195–216.
- [6] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, O’Reilly Media, 2015.
- [7] P. T. Eugster, P. A. Felber, R. Guerraoui, A.-M. Kermarrec, The many faces of publish/subscribe, *ACM Computing Surveys* 35 (2) (2003) 114–131.
- [8] A. E. Motter, Y.-C. Lai, Cascade-based attacks on complex networks, *Physical Review E* 66 (2002) 065102(R).
- [9] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley, S. Havlin, Catastrophic cascade of failures in interdependent networks, *Nature* 464 (2010) 1025–1028.
- [10] R. Albert, H. Jeong, A.-L. Barabási, Error and attack tolerance of complex networks, *Nature* 406 (2000) 378–382.
- [11] A. Avizienis, J.-C. Laprie, B. Randell, C. Landwehr, Basic concepts and taxonomy of dependable and secure computing, *IEEE Transactions on Dependable and Secure Computing* 1 (1) (2004) 11–33.
- [12] L. Bass, P. Clements, R. Kazman, *Software Architecture in Practice*, 3rd Edition, Addison-Wesley, 2012.
- [13] International Organization for Standardization, ISO/IEC 25010:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — product quality model, Tech. rep., International Organization for Standardization (2023).
- [14] International Organization for Standardization, ISO/IEC 25019:2023 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality-in-use model, Tech. rep., International Organization for Standardization (2023).
- [15] D. E. Perry, A. L. Wolf, Foundations for the study of software architecture, *ACM SIGSOFT Software Engineering Notes* 17 (4) (1992) 40–52.
- [16] R. Kazman, M. Klein, M. Barbacci, T. Longstaff, H. Lipson, J. Carriere, The architecture tradeoff analysis method, in: *Proc. 4th IEEE Int. Conf. on Engineering of Complex Computer Systems (ICECCS)*, 1998, pp. 68–78.
- [17] W. Cunningham, The WyCash portfolio management system, in: *Addendum to the Proc. Conf. on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 1992, pp. 29–30.
- [18] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Identifying architectural bad smells, in: *Proc. 13th European Conf. on Software Maintenance and Reengineering (CSMR)*, 2009, pp. 255–258.
- [19] SonarSource, Clean as you code, SonarQube documentation, <https://docs.sonarsource.com/sonarqube-server/latest/core-concepts/clean-as-you-code/introduction/> (accessed 9 September 2026) (2024).
- [20] T. J. McCabe, A complexity measure, *IEEE Transactions on Software Engineering SE-2* (4) (1976) 308–320.
- [21] S. R. Chidamber, C. F. Kemerer, A metrics suite for object oriented design, *IEEE Transactions on Software Engineering* 20 (6) (1994) 476–493.
- [22] N. Fenton, J. Bieman, *Software Metrics: A Rigorous and Practical Approach*, 3rd Edition, CRC Press, 2014.
- [23] A. Basiri, N. Behnam, R. de Rooij, L. Hochstein, L. Kosewski, J. Reynolds, C. Rosenthal, Chaos engineering, *IEEE Software* 33 (3) (2016) 35–41.
- [24] L. C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* 40 (1) (1977) 35–41.
- [25] S. Brin, L. Page, The anatomy of a large-scale hypertextual web search engine, *Computer Networks and ISDN Systems* 30 (1–7) (1998) 107–117.
- [26] U. Brandes, A faster algorithm for betweenness centrality, *Journal of Mathematical Sociology* 25 (2) (2001) 163–177.
- [27] M. E. J. Newman, *Networks: An Introduction*, Oxford University Press, 2010.
- [28] I. O. Yigit, F. Buzluca, A graph-based dependency analysis method for identifying critical components in distributed publish-subscribe systems, in: *Proc. IEEE Int. Conf. on Recent Advances in Systems Science and Engineering (RASSE)*, 2025, pp. 1–8. doi:10.1109/RASSE64831.2025.11315354.

- [29] C. Calero, M. Piattini (Eds.), *Green in Software Engineering*, Springer, Cham, Switzerland, 2015. doi:10.1007/978-3-319-08581-4.
- [30] L. Lannelongue, J. Grealey, M. Inouye, Green algorithms: Quantifying the carbon footprint of computation, *Advanced Science* 8 (12) (2021) 2100707. doi:10.1002/advs.202100707.
- [31] R. Verdecchia, J. Sallou, L. Cruz, A systematic review of Green AI, *WIREs Data Mining and Knowledge Discovery* 13 (4) (2023) e1507. doi:10.1002/widm.1507.
- [32] R. C. Cheung, A user-oriented software reliability model, *IEEE Transactions on Software Engineering SE-6* (2) (1980) 118–125.
- [33] K. Goseva-Popstojanova, K. S. Trivedi, Architecture-based approach to reliability assessment of software systems, *Performance Evaluation* 45 (2–3) (2001) 179–204.
- [34] A. Immonen, E. Niemelä, Survey of reliability and availability prediction methods from the architectural perspective, *Software and Systems Modeling* 7 (1) (2008) 49–65.
- [35] S. Becker, H. Koziol, R. Reussner, The Palladio component model for model-driven performance prediction, *Journal of Systems and Software* 82 (1) (2009) 3–22.
- [36] G. Franks, T. Al-Omari, M. Woodside, O. Das, S. Derisavi, Enhanced modeling and solution of layered queueing networks, *IEEE Transactions on Software Engineering* 35 (2) (2009) 148–161.
- [37] J. Delange, P. H. Feiler, Architecture fault modeling with the AADL error-model annex, in: *2014 40th EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA)*, IEEE, 2014, pp. 361–368. doi:10.1109/SEAA.2014.20.
- [38] Y. Gan, Y. Zhang, K. Hu, D. Cheng, Y. He, M. Pancholi, C. Delimitrou, Seer: Leveraging big data to navigate the complexity of performance debugging in cloud microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019.
- [39] Y. Gan, M. Liang, S. Dev, D. Lo, C. Delimitrou, Sage: Practical and scalable ML-driven performance debugging in microservices, in: *Proc. ACM Int. Conf. on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2021.
- [40] L. Wu, J. Tordsson, E. Elmroth, O. Kao, MicroRCA: Root cause localization of performance issues in microservices, in: *Proc. IEEE/IFIP Network Operations and Management Symposium (NOMS)*, 2020.
- [41] Z. Li, J. Chen, R. Jiao, N. Zhao, Z. Wang, S. Zhang, Y. Wu, L. Jiang, L. Yan, Z. Wang, Z. Chen, W. Zhang, X. Nie, K. Sui, D. Pei, Practical root cause localization for microservice systems via trace analysis, in: *Proc. IEEE/ACM Int. Symposium on Quality of Service (IWQoS)*, 2021.
- [42] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, D. Zhang, DeepTraLog: Trace-log combined microservice anomaly detection through graph-based deep learning, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2022.
- [43] C. Lee, T. Yang, Z. Chen, Y. Su, Y. Yang, M. R. Lyu, Eadro: An end-to-end troubleshooting framework for microservices on multi-source data, in: *Proc. IEEE/ACM Int. Conf. on Software Engineering (ICSE)*, 2023.
- [44] S. Zhang, S. Xia, W. Fan, B. Shi, X. Xiong, Z. Zhong, M. Ma, Y. Sun, D. Pei, Failure diagnosis in microservice systems: A comprehensive survey and analysis, *arXiv preprint arXiv:2407.01710* (2024).
- [45] V. R. Basili, L. C. Briand, W. L. Melo, A validation of object-oriented design metrics as quality indicators, *IEEE Transactions on Software Engineering* 22 (10) (1996) 751–761.
- [46] N. Nagappan, T. Ball, Static analysis tools as early indicators of pre-release defect density, in: *Proc. 27th Int. Conf. on Software Engineering (ICSE)*, 2005, pp. 580–586.
- [47] T. Zimmermann, R. Premraj, A. Zeller, Predicting defects for Eclipse, in: *Proc. 3rd Int. Workshop on Predictor Models in Software Engineering (PROMISE)*, 2007.
- [48] T. Menzies, J. Greenwald, A. Frank, Data mining static code attributes to learn defect predictors, *IEEE Transactions on Software Engineering* 33 (1) (2007) 2–13.
- [49] V. Bushong, D. Das, A. Al Maruf, T. Cerny, Using static analysis to address microservice architecture reconstruction, in: *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE, 2021. doi:10.1109/ASE51524.2021.9678749.
- [50] S. Schneider, A. Bakhtin, X. Li, J. Soldani, A. Brogi, T. Cerny, R. Scandariato, D. Taibi, Comparison of static analysis architecture recovery tools for microservice applications, *arXiv preprint* (2024). arXiv:2412.08352, doi:10.48550/arXiv.2412.08352.
- [51] J. Garcia, D. Popescu, G. Edwards, N. Medvidovic, Toward a catalogue of architectural bad smells, in: *Proc. 5th Int. Conf. on the Quality of Software Architectures (QoSA)*, LNCS 5581, 2009, pp. 146–162.
- [52] D. Taibi, V. Lenarduzzi, On the definition of microservice bad smells, *IEEE Software* 35 (3) (2018) 56–62.
- [53] Z. Li, P. Avgeriou, P. Liang, A systematic mapping study on technical debt and its management, *Journal of Systems and Software* 101 (2015) 193–220.
- [54] J. Humble, D. Farley, *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*, Addison-Wesley, 2010.
- [55] L. Chen, Continuous delivery: Huge benefits, but challenges too, *IEEE Software* 32 (2) (2015) 50–54.
- [56] International Organization for Standardization, ISO/IEC 25023:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of system and software product quality, Tech. rep., International Organization for Standardization (2016).
- [57] International Organization for Standardization, ISO/IEC 25021:2012 — systems and software engineering — systems and software quality requirements and evaluation (square) — quality measure elements, Tech. rep., International Organization for Standardization (2012).

- [58] T. L. Saaty, *The Analytic Hierarchy Process: Planning, Priority Setting, Resource Allocation*, McGraw-Hill, 1980.
- [59] C. Fan, L. Zeng, Y. Sun, Y.-Y. Liu, Finding key players in complex networks through deep reinforcement learning, *Nature Machine Intelligence* 2 (2020) 317–324.
- [60] C. Fan, L. Zeng, Y. Ding, M. Chen, Y. Sun, Z. Liu, Learning to identify high betweenness centrality nodes from scratch: A novel graph neural network approach, in: *Proc. 28th ACM Int. Conf. on Information and Knowledge Management (CIKM)*, 2019, pp. 559–568.
- [61] A. Varbella, K. Amara, M. El-Assady, B. Gjorgiev, G. Sansavini, PowerGraph: A power grid benchmark dataset for graph neural networks, in: *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*, 2024, arXiv:2402.02827.
- [62] T. N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2017.
- [63] W. L. Hamilton, R. Ying, J. Leskovec, Inductive representation learning on large graphs, in: *Advances in Neural Information Processing Systems 30 (NeurIPS)*, 2017, pp. 1024–1034.
- [64] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, Y. Bengio, Graph attention networks, in: *Proc. Int. Conf. on Learning Representations (ICLR)*, 2018.
- [65] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, M. Welling, Modeling relational data with graph convolutional networks, in: *Proc. European Semantic Web Conference (ESWC)*, 2018, pp. 593–607.
- [66] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, P. S. Yu, Heterogeneous graph attention network, in: *Proc. The Web Conference (WWW)*, 2019, pp. 2022–2032.
- [67] Z. Hu, Y. Dong, K. Wang, Y. Sun, Heterogeneous graph transformer, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2704–2710.
- [68] X. Fu, J. Zhang, Z. Meng, I. King, MAGNN: Metapath aggregated graph neural network for heterogeneous graph embedding, in: *Proc. The Web Conference (WWW)*, 2020, pp. 2331–2341.
- [69] G. Khodabandeh, A. Ezaz, M. Babaei, N. Ezzati-Jivan, Utilizing graph neural networks for effective link prediction in microservice architectures, in: *Proceedings of the 16th ACM/SPEC International Conference on Performance Engineering (ICPE)*, 2025.
- [70] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, GNNExplainer: Generating explanations for graph neural networks, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, 2019, pp. 9244–9255.
- [71] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang, Parameterized explainer for graph neural network, in: *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 33, 2020, pp. 19620–19631.
- [72] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*, Morgan Kaufmann, 1988.
- [73] G. Beliakov, A. Pradera, T. Calvo, *Aggregation functions: A guide for practitioners*, Studies in Fuzziness and Soft Computing 221 (2007).
- [74] R. R. Yager, On ordered weighted averaging aggregation operators in multicriteria decisionmaking, *IEEE Transactions on Systems, Man, and Cybernetics* 18 (1) (1988) 183–190.
- [75] G. H. Hardy, J. E. Littlewood, G. Pólya, *Inequalities*, 2nd Edition, Cambridge University Press, 1952.
- [76] U.S. Department of Defense, MIL-STD-498: Software development and documentation, Military standard, U.S. Department of Defense (1994).
- [77] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch geometric, in: *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.
- [78] F. Xia, T.-Y. Liu, J. Wang, W.-S. Zhang, H. Li, Listwise approach to learning to rank: Theory and algorithm, in: *Proc. 25th Int. Conf. on Machine Learning (ICML)*, 2008, pp. 1192–1199.
- [79] Team SimPy, Simpy: Discrete event simulation for Python, Software, <https://simpy.readthedocs.io> (accessed 9 September 2026) (2020).
- [80] International Organization for Standardization, ISO/IEC 25022:2016 — systems and software engineering — systems and software quality requirements and evaluation (square) — measurement of quality in use, Tech. rep., International Organization for Standardization (2016).
- [81] F. Wilcoxon, Individual comparisons by ranking methods, *Biometrics Bulletin* 1 (6) (1945) 80–83.
- [82] B. Efron, R. J. Tibshirani, *An Introduction to the Bootstrap*, Chapman & Hall, 1993.
- [83] C. Spearman, The proof and measurement of association between two things, *American Journal of Psychology* 15 (1) (1904) 72–101.
- [84] R. Schwartz, J. Dodge, N. A. Smith, O. Etzioni, Green AI, *Communications of the ACM* 63 (12) (2020) 54–63. doi: 10.1145/3381831.
- [85] E. Strubell, A. Ganesh, A. McCallum, Energy and policy considerations for deep learning in NLP, in: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, Florence, Italy, 2019, pp. 3645–3650. doi: 10.18653/v1/P19-1355.
- [86] D. Patterson, J. Gonzalez, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, J. Dean, Carbon emissions and large neural network training, arXiv preprint arXiv:2104.10350 (2021). doi: 10.48550/arXiv.2104.10350.
- [87] S. Georgiou, M. Kechagia, T. Sharma, F. Sarro, Y. Zou, Green AI: Do deep learning frameworks have different costs?, in: *Proceedings of the 44th International Conference on Software Engineering (ICSE)*, 2022, pp. 1082–1094. doi: 10.1145/3510003.3510221.
- [88] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, V. Prasanna, GraphSAINT: Graph sampling based inductive engine, in: *Proc. International Conference on Learning Representations (ICLR)*, 2020.
- [89] I. O. Yigit, F. Buzluca, [dataset] software-as-a-graph: Replication package (datasets, generator configurations, simulation

1529 harnesses, model checkpoints, and analysis scripts), <https://doi.org/10.5281/zenodo.14922108> (2026). doi:10.5281/
1530 **zenodo.14922108**.